VIETNAM NATIONAL UNIVERSITY

HO CHI MINH CITY UNIVERSITY
OF TECHNOLOGY

CAPSTONE PROJECT 1

Design and FPGA Implementation of an Enhanced Traffic Light Controller with Pedestrian Request Handling and Adaptive Buzzer Feedback

Instructor: Ph.D. Nguyen Ly Thien Truong

Student: Pham Thanh Vinh

ID: 2351075

Ho Chi Minh City – May 2026

Acknowledgement

I would like to express my sincere gratitude to my supervisor, Ph.D.Nguyen Ly Thien Truong, for his valuable guidance, support, and encouragement throughout this capstone project. At the beginning of the project, I was still uncertain about my technical direction and was considering different possibilities. Ph.D. Truong's advice helped me gain a clearer orientation toward digital system design, which became an important foundation for the development of this project. His guidance not only helped me define a suitable project direction but also encouraged me to approach the work with a stronger engineering mindset.

I am especially thankful for his dedicated weekly supervision, where he provided constructive feedback, pointed out weaknesses in my design, and suggested new technical directions for improvement. Through these discussions, I was able to refine the traffic light controller, correct design mistakes, and develop the original idea into a working FPGA-based system. His comments and suggestions played an important role in helping me understand the relationship between finite state machines, timer control, hardware peripherals, FPGA implementation, and practical verification.

I would also like to thank the lecturers and staff members at Laboratory 203B3 for providing me with access to the FPGA development kit and a working space during the project implementation. Their support created a favorable environment for me to carry out the design, simulation, synthesis, and hardware testing stages of the project.

In addition, I would like to thank several of my friends who shared useful ideas and suggestions during the early development of this project. In particular, their suggestion to add a pedestrian button feature helped make the traffic light controller more practical and closer to a real-world traffic system. Although this project was completed individually, these discussions gave me additional perspectives and motivation to improve the functionality of the design.

Finally, I would like to express my appreciation to everyone who supported me during the completion of this capstone project. This work has been a valuable learning experience, allowing me to strengthen my understanding of FPGA-based digital design, finite state machine implementation, hardware verification, Quartus synthesis, and practical system integration on an FPGA development board.

Abstract

This capstone project presents the design, verification, and FPGA implementation of an enhanced traffic light controller on the DE10-Standard FPGA development board. The final working system is implemented using a finite state machine based architecture with a phase timer, pedestrian request logic, seven-segment display modules, GPIO traffic light outputs, and an adaptive red-phase buzzer feedback module. The purpose of the project is to demonstrate a practical FPGA-based digital control system that can be simulated, synthesized, and tested on real hardware.

The controller operates through a normal traffic sequence consisting of a 15-second green phase, a 3-second yellow phase, and a 15-second red phase. A pedestrian request button is included to make the system more interactive. When a request is detected during the green phase while more than 10 seconds remain, the controller shortens the remaining green time to 2 seconds. When a request is detected during the red phase while fewer than 5 seconds remain, the controller extends the red phase by 10 seconds. These functions are implemented using a traffic FSM, a pedestrian request register, and a phase timer capable of loading, adding, and counting down timer values.

The system also provides visual and audio feedback for demonstration. The countdown value and pedestrian request status are displayed on the seven-segment displays. External red, yellow, and green LEDs are driven through GPIO[0], GPIO[1], and GPIO[2], while the buzzer driver is connected to GPIO[3]. During the red phase, the buzzer changes its beep rate according to the remaining red time. The design was described in SystemVerilog, verified using ModelSim simulation, synthesized using Intel Quartus Prime, and successfully tested on the DE10-Standard board. Overall, the project provides practical experience in FSM design, timer control, FPGA implementation, GPIO interfacing, and hardware verification.

Table of Contents

List of Figures

List of Tables

Chapter 1. Introduction

1.1 Background and Motivation

Traffic light control is a representative application of real-time digital system design. A traffic controller must manage the order and duration of light phases while responding to external events such as pedestrian requests. Because the system combines sequential control, timing logic, input processing, output driving, and hardware interfacing, it is suitable for a capstone project in FPGA-based digital design.

This project implements an enhanced traffic light controller on the DE10-Standard FPGA development board using SystemVerilog. The controller is based on a finite state machine architecture and is extended with additional practical functions, including pedestrian request handling, countdown display, manual timer loading for testing, external LED outputs, and buzzer feedback. The final system was synthesized using Intel Quartus Prime and tested on the actual FPGA board.

The motivation of the project is to build a working digital controller that is more complete than a basic green-yellow-red FSM. Instead of stopping at simple LED sequencing, the design integrates several supporting modules such as a button pulse generator, pedestrian request register, phase timer, seven-segment display driver, and red-phase buzzer module. This makes the project closer to a practical traffic light demonstration system and provides meaningful experience in FPGA design, simulation, synthesis, and hardware debugging.

1.2 Problem Statement

A basic FSM-based traffic light controller can generate a fixed sequence of red, yellow, and green lights. However, a practical demonstration system should also handle user input, display timing information, and provide visible or audible feedback. The main problem of this project is therefore to design a synthesizable FPGA traffic light controller that can operate correctly on real hardware while supporting additional functions beyond the basic phase sequence.

The controller must maintain the normal phase order, process pedestrian requests according to timing conditions, update the countdown timer accurately, and drive the required board-level outputs. In addition, the system must interface safely with external LED and buzzer driver circuits through GPIO pins. The design must be modular enough for simulation and debugging, but not unnecessarily complicated for FPGA implementation.

Another important challenge is timing coordination. The FPGA board provides a 50 MHz clock, while the traffic phases must change according to real-time second-level delays. Therefore, the design uses the 50 MHz clock for synchronous logic and generates a one-second tick signal as an enable pulse for the phase timer. This approach keeps the system synchronous and avoids using the one-second tick as a separate clock domain.

1.3 Project Objectives

The main objective of this project is to design, simulate, synthesize, and demonstrate an enhanced FPGA-based traffic light controller on the DE10-Standard board. The specific objectives are as follows:

1. Implement the normal traffic light cycle with green, yellow, and red phases.

2. Generate an accurate countdown timer using a one-second tick derived from the 50 MHz system clock.

3. Handle pedestrian requests during the green and red phases according to predefined timing rules.

4. Provide countdown information on seven-segment displays for easier observation during testing.

5. Drive external red, yellow, and green LEDs through GPIO pins on the DE10-Standard board.

6. Drive a buzzer through an external driver circuit to provide red-phase audio feedback.

7. Verify the system using simulation and validate the final design on the FPGA development board.

1.4 Project Scope and Limitations

The scope of this project is limited to a single-direction traffic light controller. The system controls one set of red, yellow, and green lights and does not implement a complete two-road intersection. This scope is appropriate for demonstrating the main digital design concepts while keeping the implementation manageable for FPGA testing.

The pedestrian request is generated by a push button rather than by an automatic sensor or camera. When a request is accepted during the green phase, the remaining green time is shortened. When a request is accepted near the end of the red phase, the red duration is extended. These rules are implemented directly in the traffic FSM and phase timer logic.

The buzzer output is implemented as a simple GPIO-controlled beep signal suitable for an active buzzer or an external buzzer driver circuit. The design does not generate a complex audio-frequency waveform for a passive buzzer. In addition, the manual timer input is mainly used as a testing and extension feature rather than the main operating mode of the traffic controller.

1.5 Report Organization

This report is organized as follows. Chapter 1 introduces the background, motivation, problem statement, objectives, scope, and limitations of the project. Chapter 2 presents the technical background related to FPGA design, finite state machines, timers, seven-segment displays, and GPIO interfacing.

Chapter 3 describes the system requirements and design specification. Chapter 4 presents the overall system architecture and module-level data flow. Chapter 5 explains the SystemVerilog

implementation of the main modules, including the traffic FSM, phase timer, pedestrian request register, display modules, and buzzer controller.

Chapter 6 discusses the simulation and verification process. Chapter 7 summarizes the FPGA synthesis results, resource utilization, timing considerations, and compilation warnings. Chapter 8 presents the hardware implementation and demonstration results on the DE10-Standard board. Chapter 9 concludes the report and suggests possible future improvements.

Chapter 2. Technical Background

2.1 FPGA-Based Digital System Design

A Field-Programmable Gate Array (FPGA) is a reconfigurable digital device that allows designers to implement custom hardware circuits using hardware description languages such as Verilog or SystemVerilog. Unlike a microcontroller, which executes software on a fixed processor, an FPGA implements hardware modules that can operate concurrently. This makes it suitable for systems that require precise timing, parallel signal processing, and direct control of input and output devices.

In this project, the DE10-Standard FPGA board is used as the target platform. The design is described in SystemVerilog, simulated using ModelSim, synthesized using Intel Quartus Prime, and finally demonstrated on the FPGA board. The system uses the 50 MHz board clock as the main clock source. A one-second tick is generated from this clock and used to update the countdown timer.

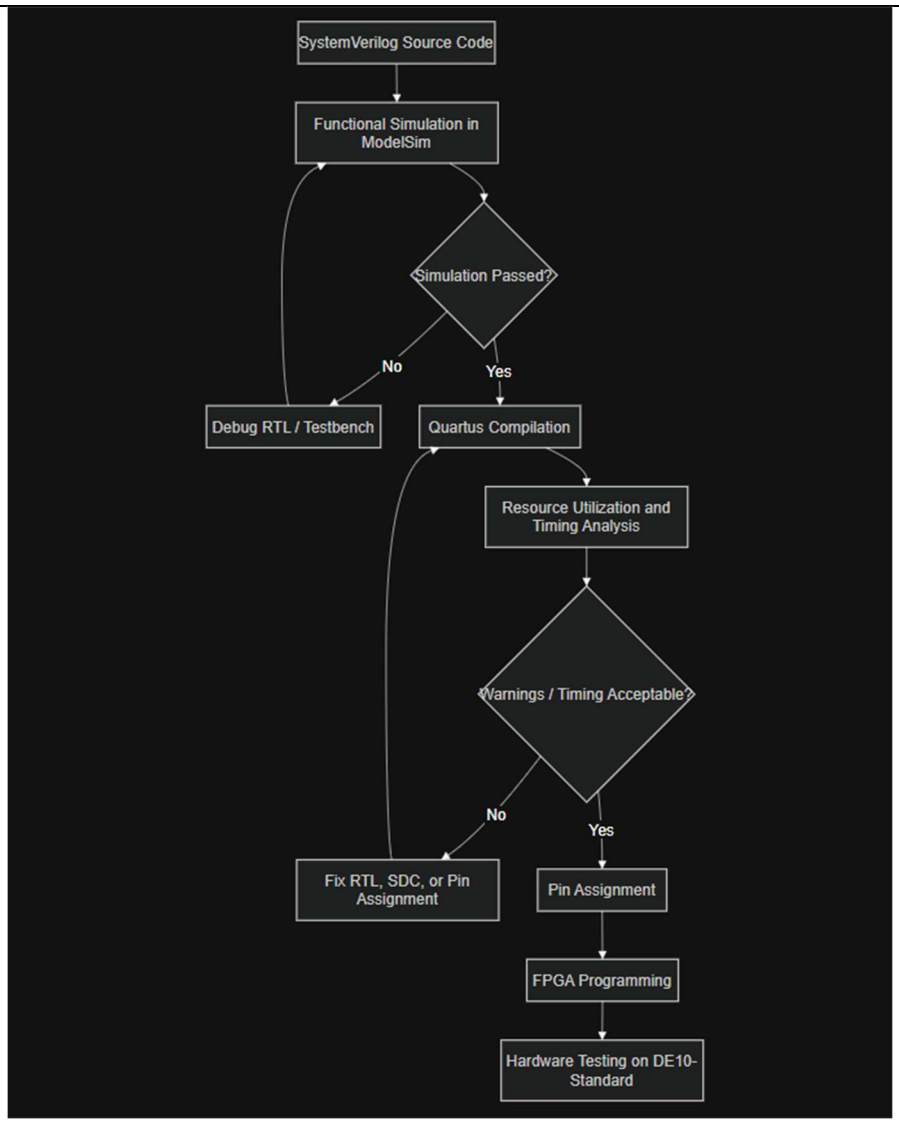


Figure 2.1. FPGA design flow used in the project.

2.2 Finite State Machine for Traffic Light Control

A finite state machine (FSM) is a common method for designing control logic. A traffic light controller is naturally suitable for FSM design because it operates through a limited number of states, such as green, yellow, and red phases. State transitions are determined by timer expiration, pedestrian requests, and other control conditions.

The final working design uses an FSM as the main traffic phase controller. The FSM controls the normal sequence of green, yellow, and red phases. It also includes special states for shortening the green phase and extending the red phase when a valid pedestrian request is detected. This approach is simple, deterministic, and suitable for hardware implementation on FPGA.

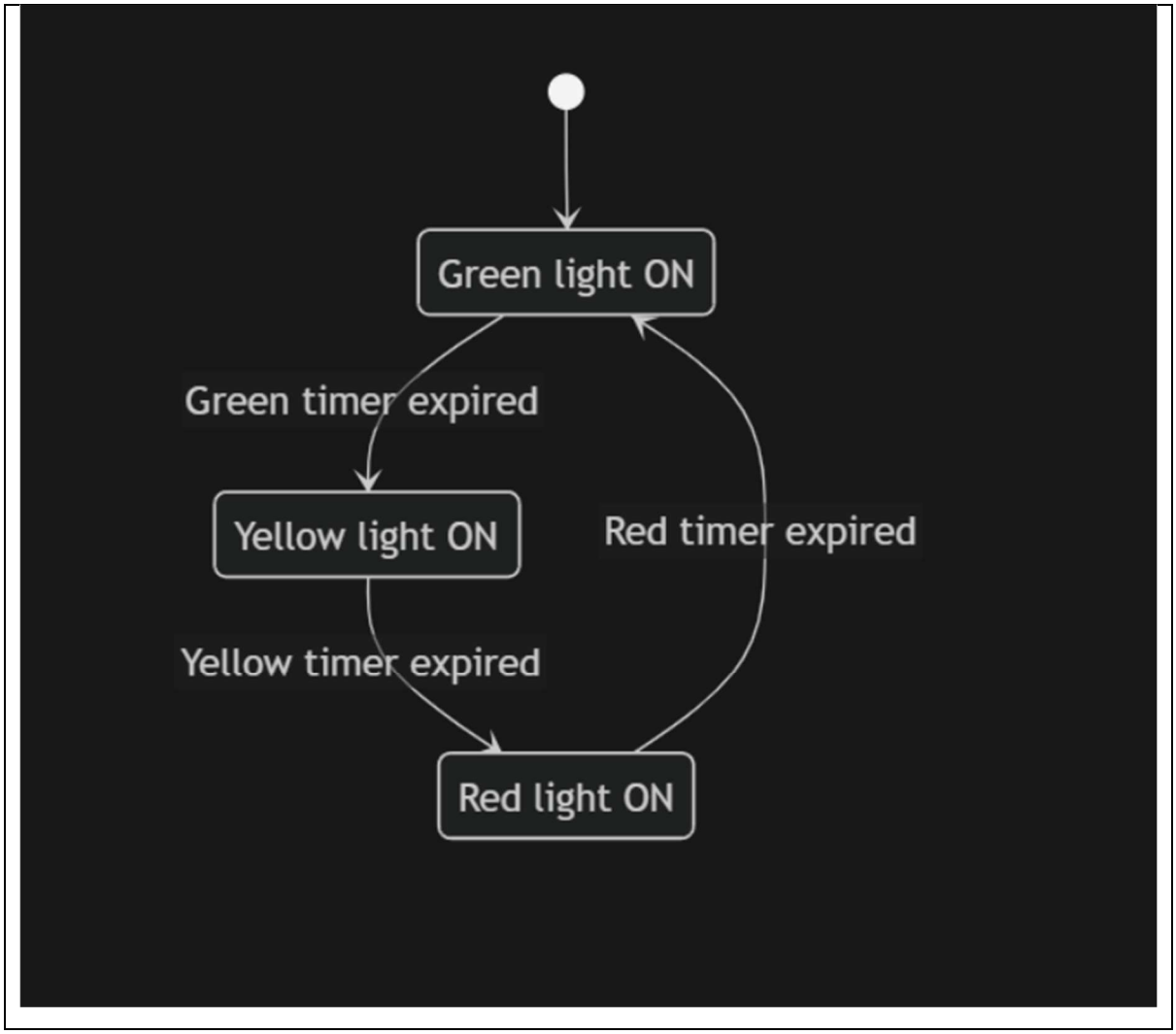


Figure 2.2. Basic FSM concept for traffic light control.

2.3 Button Debouncing and Pulse Generation

Mechanical push buttons do not always produce a clean digital transition. When pressed or released, the contact may bounce and generate several short transitions before

becoming stable. If this signal is connected directly to an FSM, one physical button press may be detected as multiple requests.

To avoid this issue, the design uses a button pulse module. The input button signal is filtered using a debounce counter and then converted into a one-clock pulse. The pedestrian request button and manual load input both use this approach. The pulse output is easier for the traffic controller to process and prevents repeated triggering from one button press.

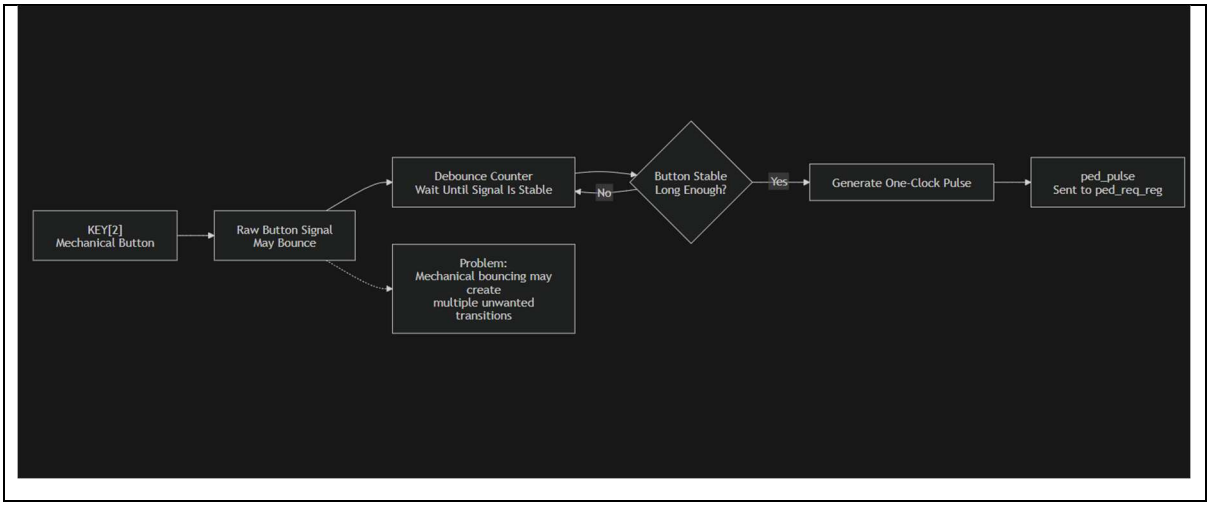


Figure 2.3. Button debounce and pulse generation concept.

2.4 Countdown Timer Design

A traffic light controller requires a countdown timer to determine when each phase should finish. In this project, the timer is implemented in the phase timer module. The timer can load a new value, add a value, subtract a value, or count down once per second. These functions are controlled by the traffic FSM.

The timer receives a one-second tick generated by the clock divider. When the system is enabled and no load or add operation is requested, the timer count decreases by one on each tick until it reaches zero. The timer also generates status flags such as timer expired, ten-second remaining, and five-second remaining. These flags are used by the FSM to decide phase transitions and pedestrian request behavior.

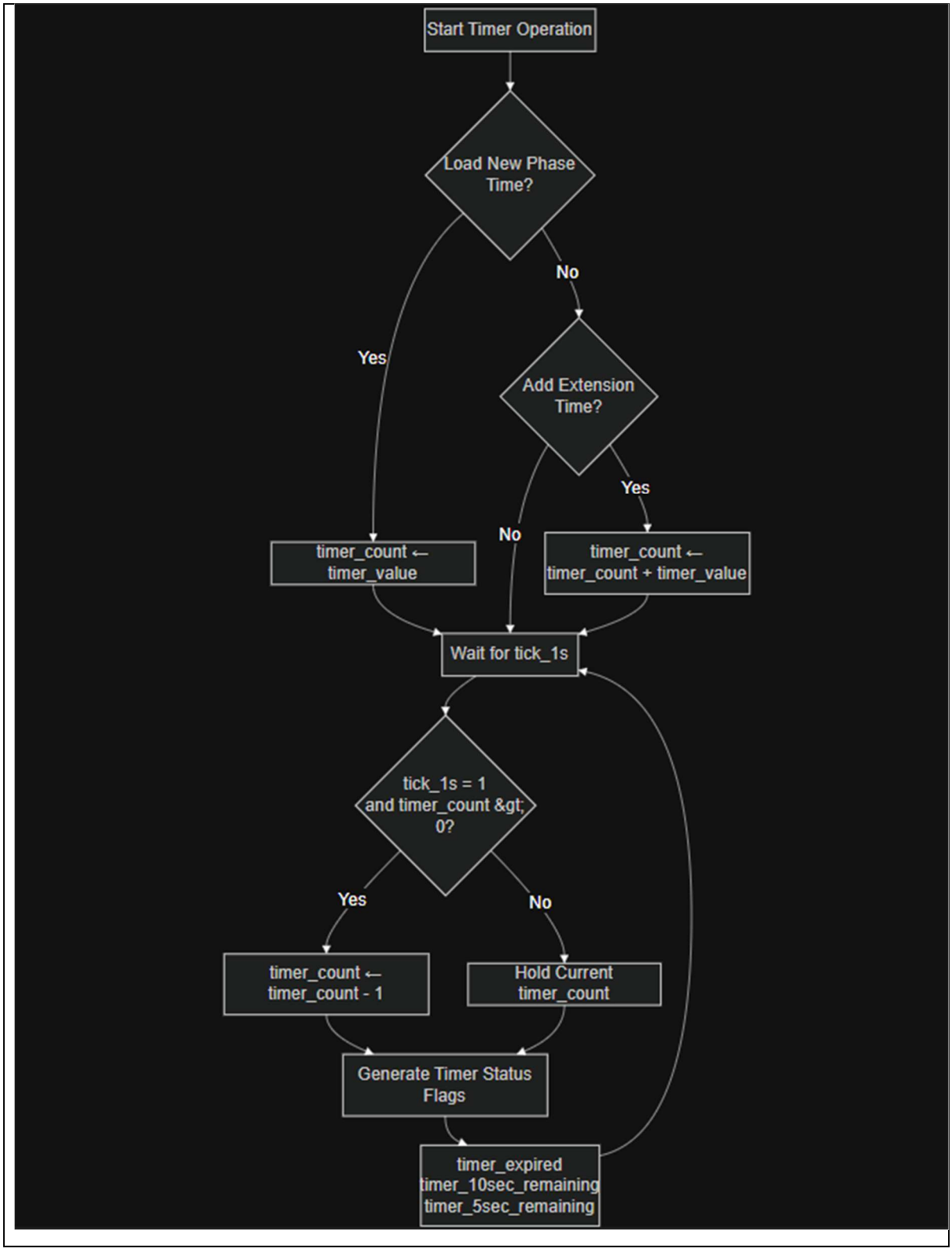


Figure 2.4. Countdown timer operation concept.

2.5 Seven-Segment Display and GPIO Output

The DE10-Standard board provides seven-segment displays and general-purpose input/output pins that can be used to observe and demonstrate the controller behavior.

The countdown value is shown on the seven-segment displays, while debug LEDs and GPIO pins are used to show the traffic light states and buzzer signal.

The external hardware demonstration uses three LEDs and a buzzer driver circuit on a breadboard. The FPGA provides digital control signals through GPIO pins. The LED outputs represent the red, yellow, and green traffic lights, while the buzzer output provides audio feedback during the red phase. The buzzer signal is connected through a driver circuit rather than being driven directly as a high-current load from the FPGA pin.

Table 2.1. Key digital design concepts used in the project.

Concept	Role in This Project
FSM	Controls the traffic phase sequence and pedestrian request handling
Clock divider	Generates one-second tick from the 50 MHz system clock
Phase timer	Stores and updates the current countdown value
Button pulse	Debounces push buttons and generates one-clock pulses
Seven-segment display	Shows countdown and status information
GPIO	Drives external LEDs and buzzer driver circuit

Chapter 3. System Requirements and Design Specification

3.1 Functional Requirements

The system is designed as an enhanced FPGA-based traffic light controller. It must perform the normal green-yellow-red traffic sequence, respond to pedestrian requests, display useful information on the board, and drive external hardware through GPIO. The main functional requirements are listed in Table 3.1.

Table 3.1. Functional requirements of the final system.

ID	Requirement	Description
FR1	Normal phase cycle	The controller shall operate through green, yellow, and red phases.
FR2	Countdown timer	The current phase time shall be displayed and counted down once per second.
FR3	Green phase request handling	A valid pedestrian request during green shall shorten the remaining green time to 2 seconds.
FR4	Red phase request handling	A valid pedestrian request near the end of red shall extend the red phase by 10 seconds.
FR5	Pedestrian request latch	A short button press shall be stored until the request is accepted and cleared.
FR6	External LEDs	Red, yellow, and green outputs shall be sent to external LEDs.
FR7	Buzzer feedback	A buzzer shall beep during the red phase with different rates based on remaining time.
FR8	Simulation and hardware test	The design shall be verified by simulation and demonstrated on the FPGA board.

3.2 Input and Output Specification

The board-level inputs and outputs are connected through the top-level wrapper module. The main input clock is CLOCK_50. KEY[0] is used as the active-low reset

input. KEY[2] is used as the pedestrian request button. SW[9] is used as the enable signal, and SW[7:0] may be used as manual time input depending on the selected test configuration.

The traffic light outputs are displayed on debug LEDs and connected to external LED circuits through GPIO. The countdown value is displayed on the seven-segment displays. The buzzer output is connected to an external driver circuit on the breadboard.

Table 3.2. Board-level input and output specification.

Signal	Direction	Function
CLOCK_50	Input	50 MHz system clock
KEY[0]	Input	Active-low reset
KEY[2]	Input	Pedestrian request button
SW[9]	Input	System enable
SW[7:0]	Input	Manual time value or debug input
LEDR[2:0]	Output	Debug indication for green, yellow, and red outputs
HEX displays	Output	Countdown or status display
GPIO[0]	Output	External red LED
GPIO[1]	Output	External yellow LED
GPIO[2]	Output	External green LED
GPIO[3]	Output	Buzzer driver input

3.3 Timing Requirements

The system uses the 50 MHz onboard clock for all sequential logic. A clock divider generates a one-second tick for the timer. The traffic phase durations are fixed in the final demonstration: green lasts 15 seconds, yellow lasts 3 seconds, and red lasts 15 seconds during normal operation.

The pedestrian request modifies the normal timing only under specific conditions. During the green phase, if more than 10 seconds remain, the controller forces the remaining green time to 2 seconds. During the red phase, if fewer than 5 seconds remain, the controller adds 10 seconds to the red timer. Requests during unsafe or unsupported conditions do not disturb the normal phase sequence.

Table 3.3. Timing behavior of the traffic controller.

Condition	Timer Action
Green initialization	Load timer with 15 seconds
Yellow initialization	Load timer with 3 seconds
Red initialization	Load timer with 15 seconds
Valid request during green	Force remaining green time to 2 seconds

Valid request during red	Add 10 seconds to current red timer
One-second tick	Decrease timer by 1 if timer is greater than zero

3.4 Hardware Requirements

The hardware demonstration requires the DE10-Standard FPGA board, a breadboard circuit with three LEDs, current-limiting resistors, and a buzzer driver circuit. The FPGA GPIO pins provide logic-level control signals. The LED circuits use the GPIO outputs to indicate the red, yellow, and green phases. The buzzer driver receives the GPIO buzzer control signal and drives the buzzer safely.

A common ground must be shared between the FPGA board and the external breadboard circuit. The buzzer should not be connected as a high-current load directly to an FPGA pin. Instead, a driver stage is used so that the FPGA only provides the control signal.

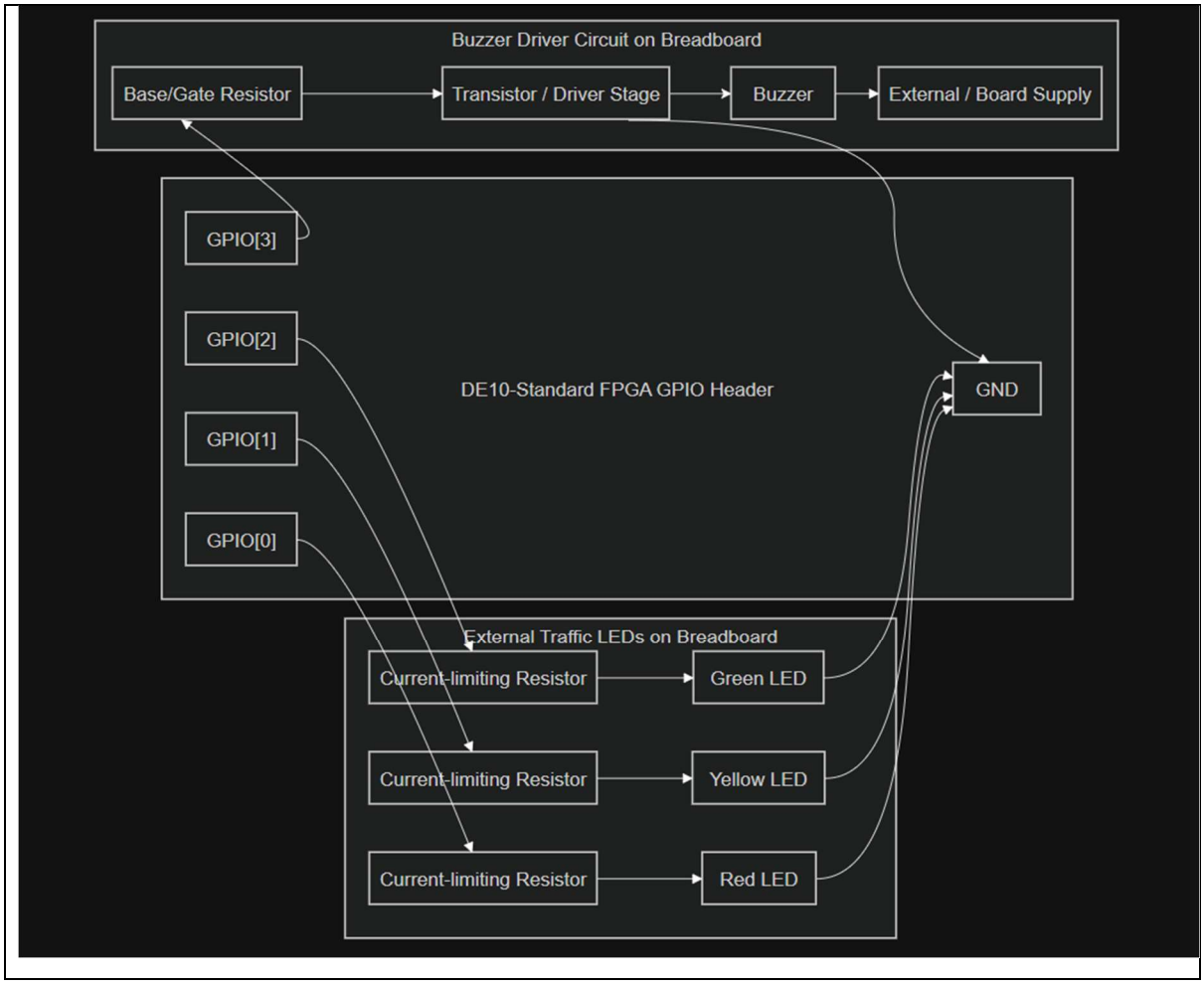


Figure 3.1. External LED and buzzer hardware connection.

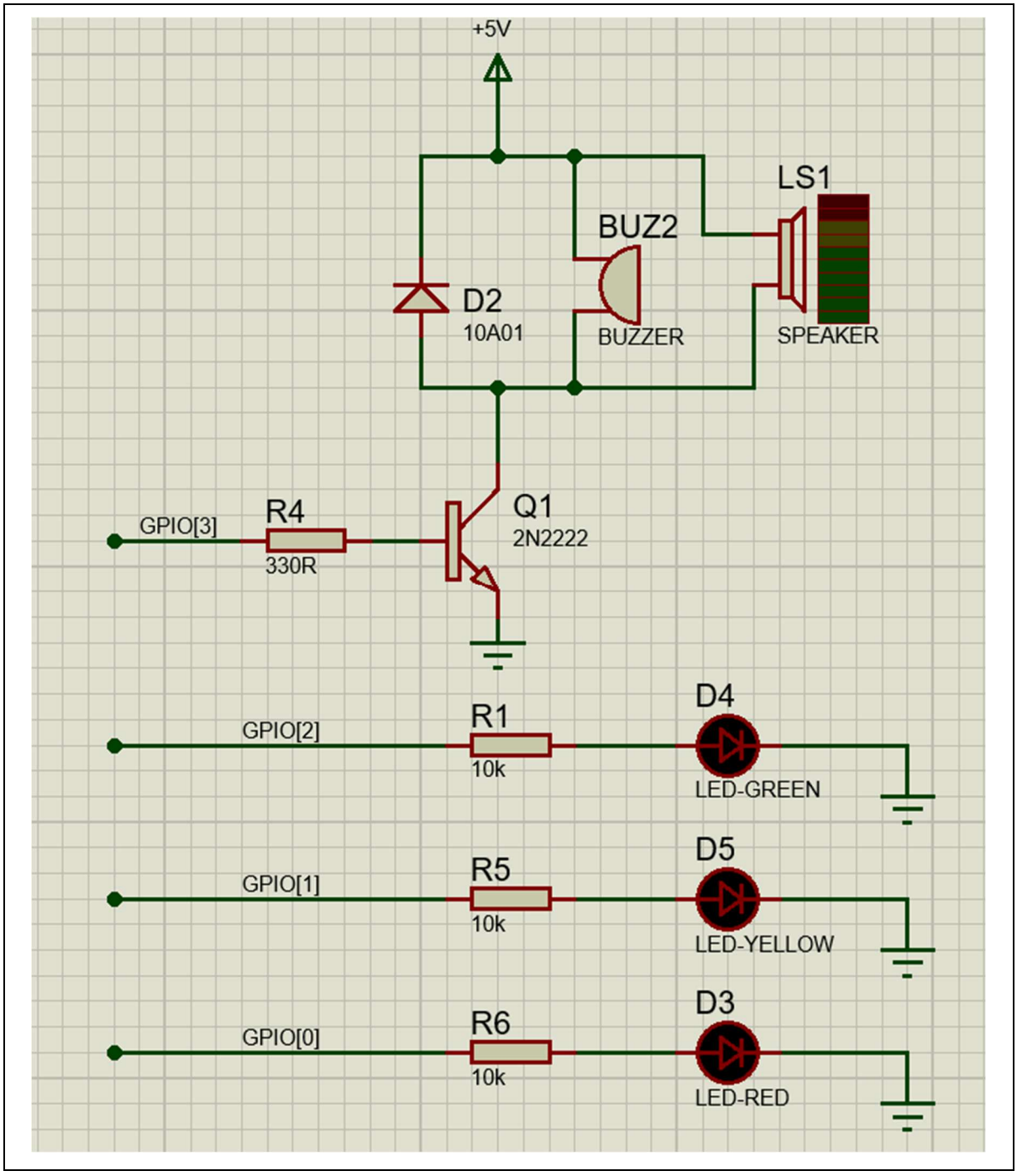


Figure 3.2. Schematic of breadboard circuit for LEDs and buzzer driver.

Chapter 4. System Architecture

4.1 Overall Block Diagram

The final system is organized as a modular FSM-based traffic light controller. The top-level wrapper connects the FPGA board signals to the main controller and external output modules. The main controller contains the clock divider, button pulse modules, pedestrian request register, traffic FSM, phase timer, and seven-segment display driver. Additional modules are used for status display and red-phase buzzer feedback when those outputs are included in the final hardware setup.

The architecture is designed around a clear control path. The pedestrian button is first debounced and converted to a pulse. This pulse sets the pedestrian request register. The traffic FSM observes the request and timer status flags, then generates LED signals and timer control commands. The phase timer updates the countdown value and sends status flags back to the FSM.

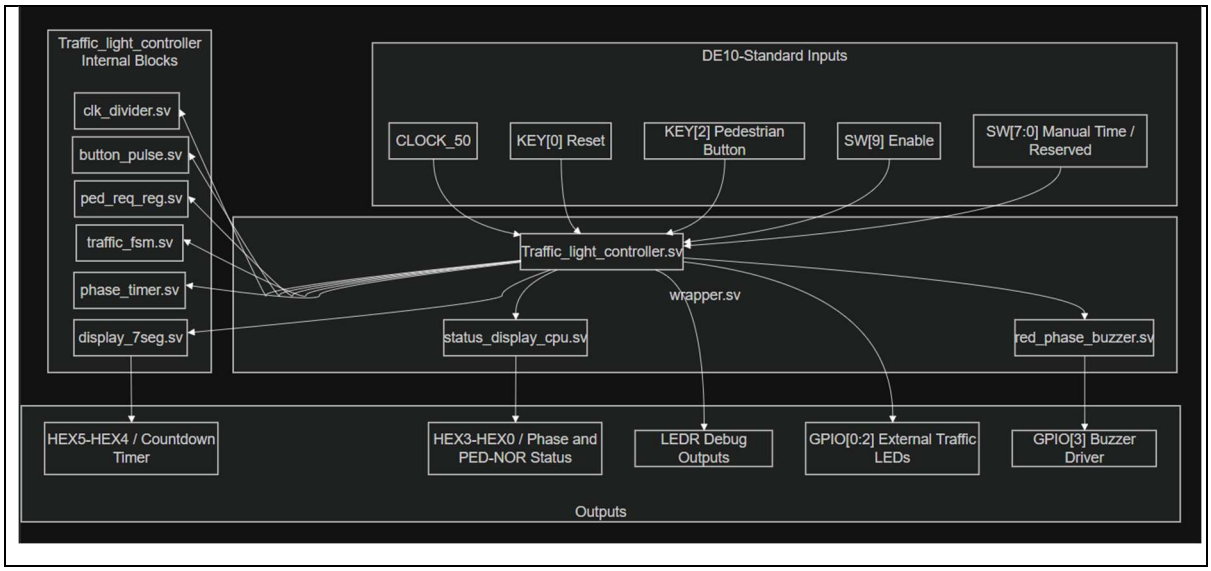


Figure 4.1. Overall block diagram of the final FSM-based traffic light controller.

4.2 Main Data and Control Flow

The data and control flow begins with the board inputs. The reset, enable, pedestrian button, and manual time switches enter the wrapper module. The pedestrian button is passed through the button pulse module before being stored in the pedestrian request register. The request remains active until the FSM accepts it and generates the request clear signal.

The traffic FSM is the central decision block. It receives the current pedestrian request and timer status flags. Based on these inputs and the current state, it controls the active traffic light output and sends commands to the phase timer. The phase timer then loads, adds, subtracts, or counts down the timer value. The current timer value is sent to the seven-segment display and is also used by the buzzer module to select the beep rate.

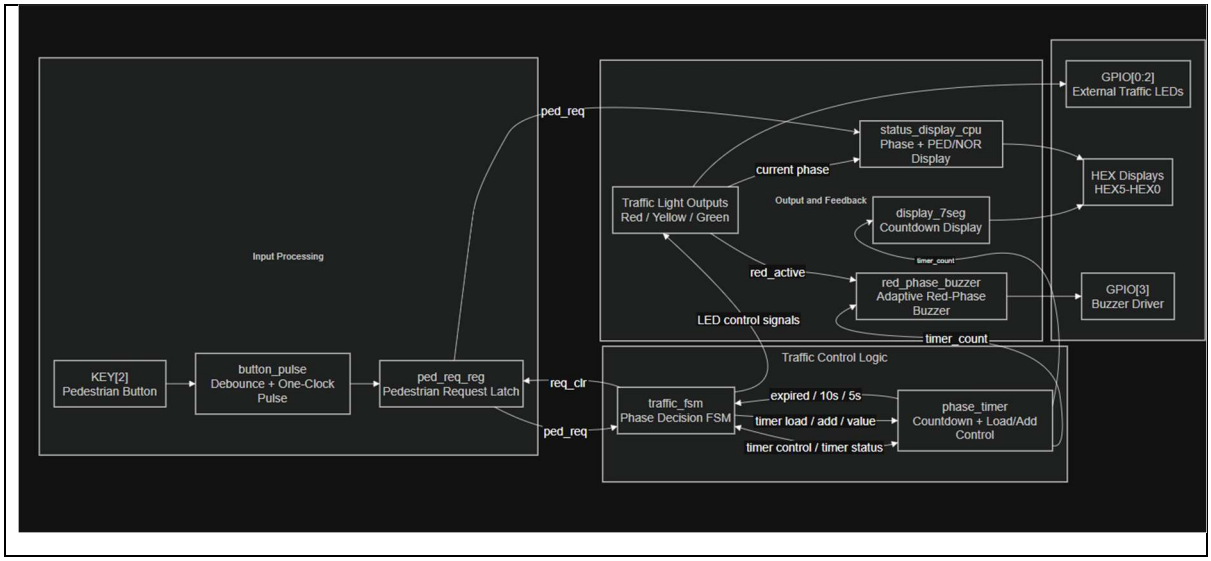


Figure 4.2. Control and data flow between button, FSM, timer, display, and buzzer modules.

4.3 Top-Level Module Hierarchy

The SystemVerilog implementation is separated into small modules. This hierarchy makes the design easier to test and modify. The top-level hierarchy used in the final project is summarized in Table 4.1.

Table 4.1. Main SystemVerilog modules and their functions.

Module	Function
wrapper	Top-level board interface and IO mapping
Traffic_light_controller	Main controller core connecting all internal modules
clk_divider	Generates one-second tick from the 50 MHz clock
button_pulse	Debounces button input and generates one-clock pulse
ped_req_reg	Stores pending pedestrian request
traffic_fsm	Controls traffic states and timer commands
phase_timer	Maintains countdown value and timer status flags
display_7seg	Displays timer value on seven-segment displays
hexled	Converts decimal digit to seven-segment code
status_display_cpu	Displays phase and pedestrian status when used
red_phase_buzzer	Generates adaptive buzzer pattern during red phase

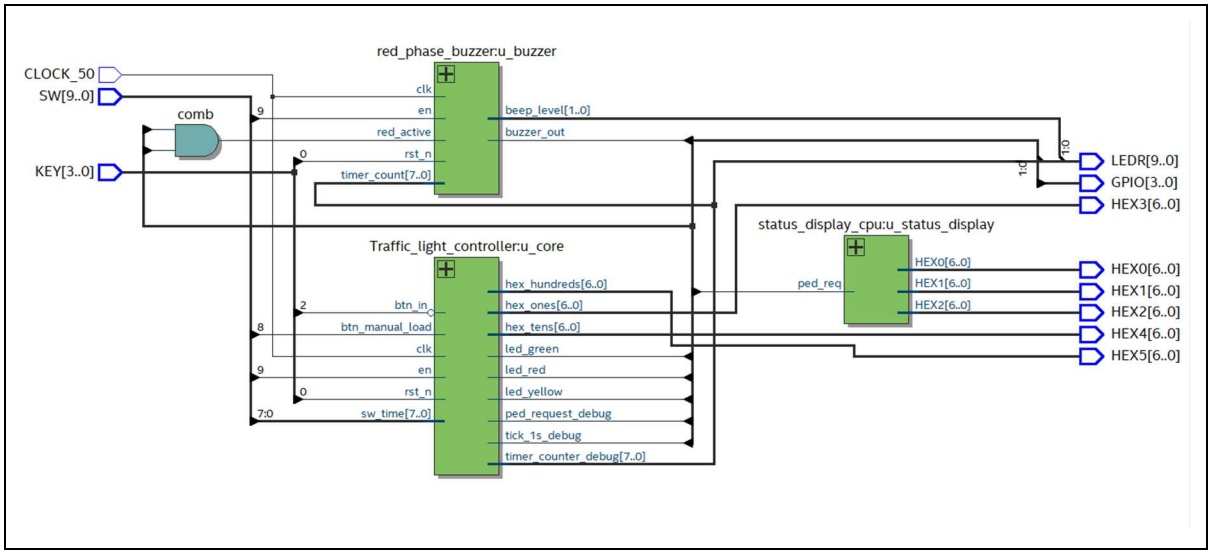


Figure 4.3. Quartus RTL hierarchy of the final design.

Chapter 5. SystemVerilog Module Implementation

5.1 Top-Level Wrapper Module

The wrapper module is the top-level entry of the FPGA design. It maps the physical DE10-Standard board signals to the internal controller. The module receives CLOCK_50, KEY, and SW inputs, and outputs LEDR, seven-segment display signals, and GPIO signals for the external circuit.

Inside the wrapper, KEY[0] is used as the active-low reset signal, SW[9] is used as the enable signal, and KEY[2] is inverted to generate the pedestrian button input. The wrapper instantiates the main Traffic_light_controller module and maps the controller outputs to board-level displays and debug signals.

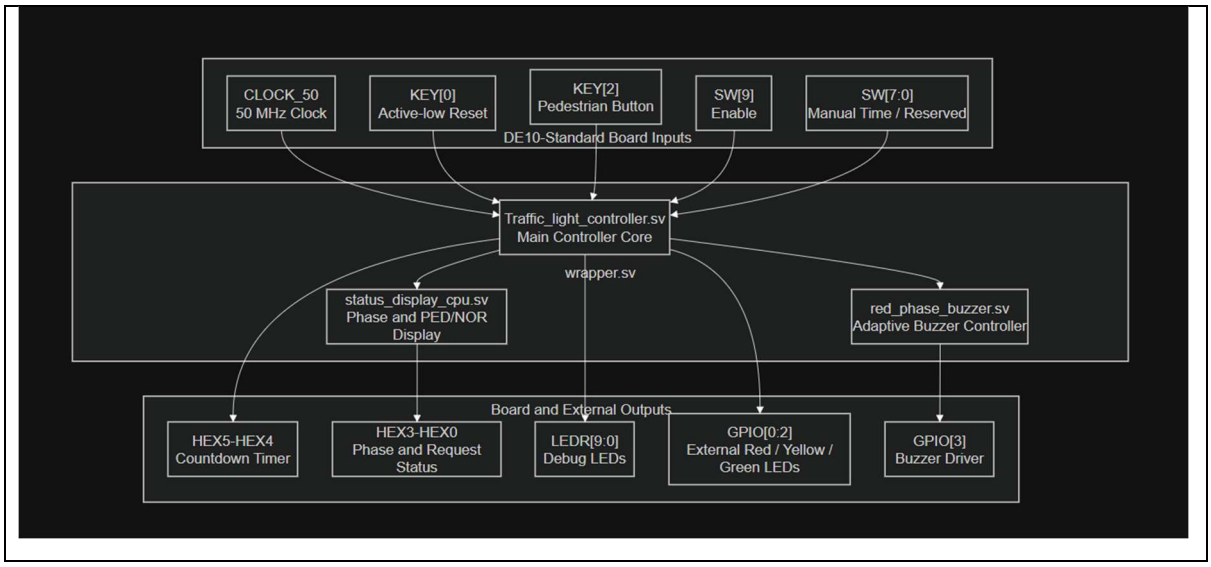


Figure 5.1. Board-level wrapper connection.

5.2 Main Traffic Controller Module

The Traffic_light_controller module connects the internal functional blocks of the traffic light system. It instantiates the clock divider, button pulse modules, pedestrian request register, traffic FSM, phase timer, and countdown display module. This module forms the main controller core below the board-level wrapper.

The internal signals include the one-second tick, button pulse, pedestrian request, request clear signal, timer control signals, timer value, timer count, and timer status flags. These signals form the connection between the FSM and the phase timer. The controller also forwards the timer count and tick signal to debug outputs for testing.

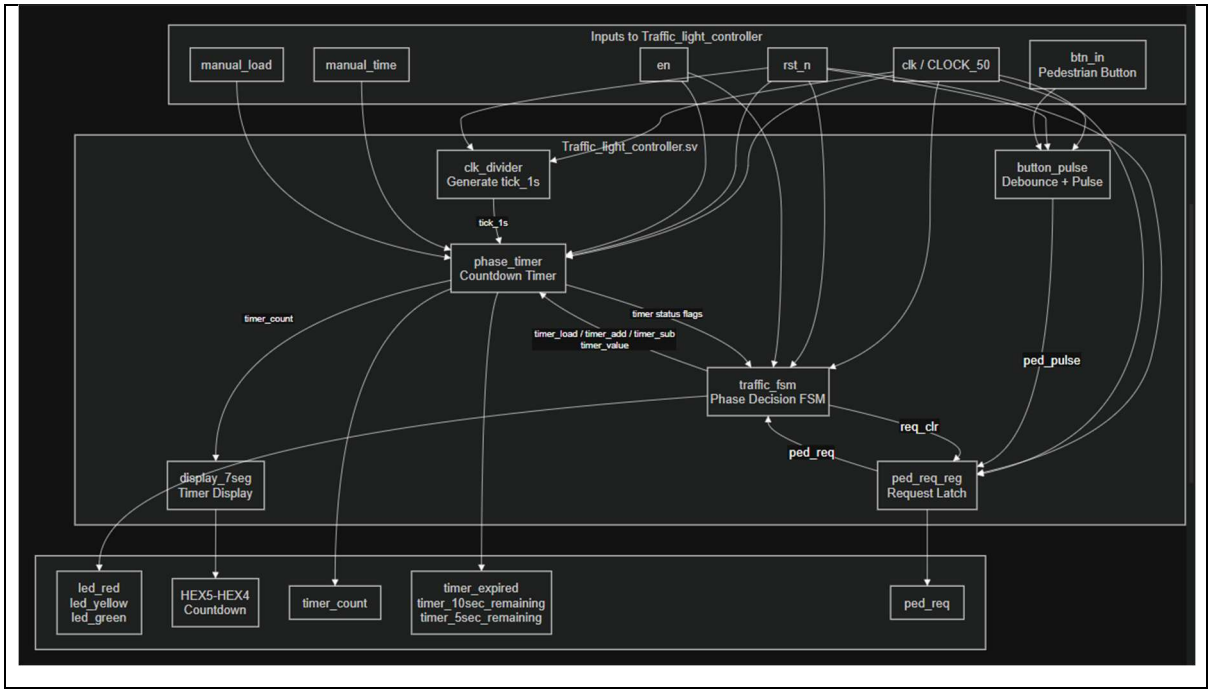


Figure 5.2.1. Internal structure of the Traffic_light_controller module.

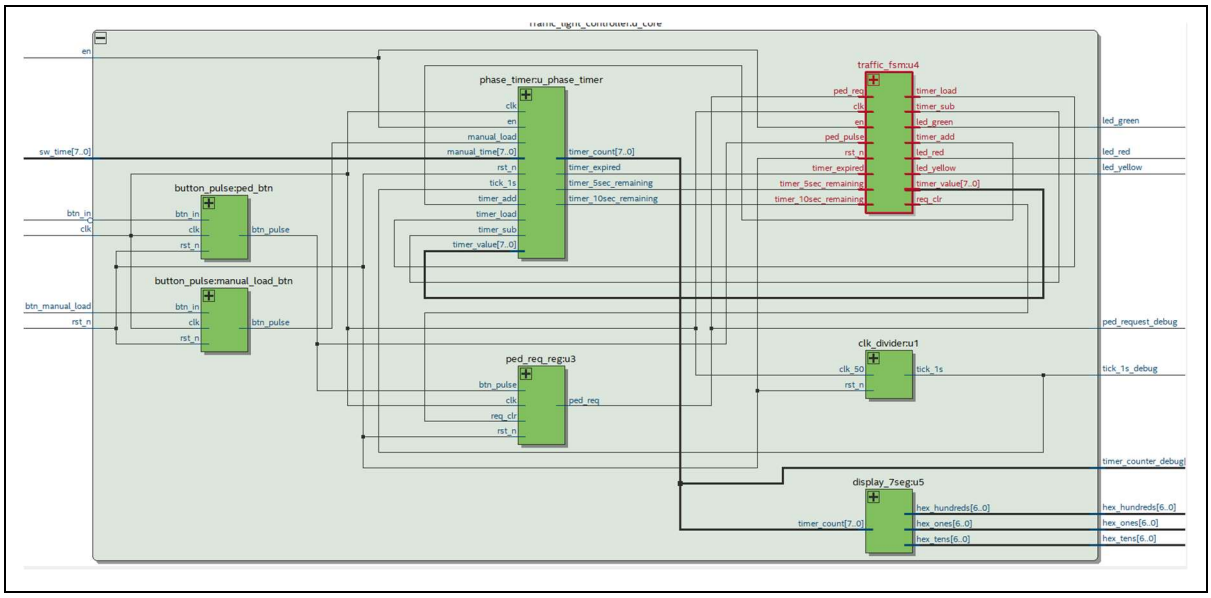


Figure 5.2.2. Internal structure of the Traffic_light_controller module in RTL Viewer.

5.3 Traffic FSM Module

The traffic_fsm module controls the traffic phase sequence. The FSM contains the IDLE, GREEN_INIT, GREEN, YELLOW_INIT, YELLOW, RED_INIT, RED, GREEN_DEC, and RED_INC states. Initialization states are used to load the timer with the correct phase duration. Running states wait for timer expiration or valid pedestrian request conditions.

During GREEN_INIT, the FSM turns on the green output and loads the timer with 15 seconds. During YELLOW_INIT, it turns on the yellow output and loads the timer

with 3 seconds. During RED_INIT, it turns on the red output and loads the timer with 15 seconds. GREEN_DEC forces the remaining green time to 2 seconds, while RED_INC adds 10 seconds to the red timer. The FSM also generates req_clr when a pedestrian request has been accepted.

Table 5.1. Traffic FSM states and output actions.

State	Main Output Action
IDLE	All light outputs off; timer inactive
GREEN_INIT	Green output on; load timer with 15 seconds
GREEN	Green output remains on; wait for timer expiration or valid request
YELLOW_INIT	Yellow output on; load timer with 3 seconds
YELLOW	Yellow output remains on; pedestrian request ignored
RED_INIT	Red output on; load timer with 15 seconds
RED	Red output remains on; wait for timer expiration or valid request
GREEN_DEC	Clear request and force green timer to 2 seconds
RED_INC	Clear request and add 10 seconds to red timer

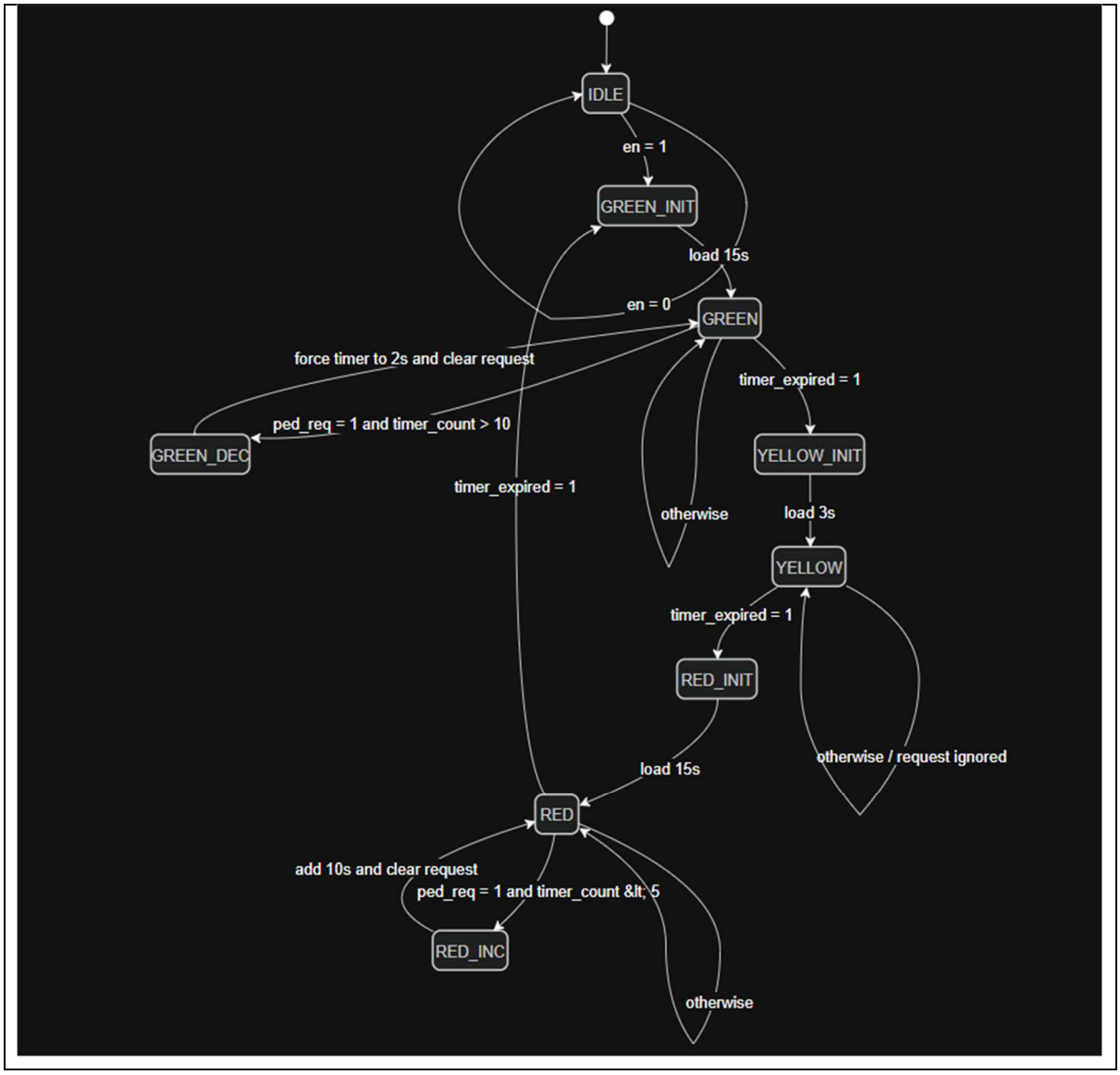


Figure 5.3.1. Traffic FSM state diagram.

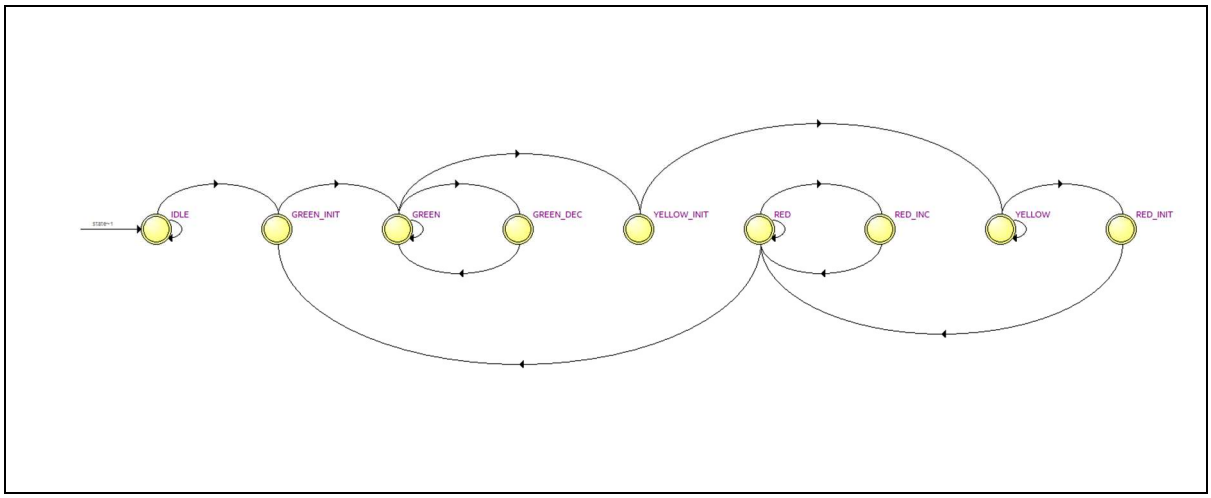


Figure 5.3.2. Traffic FSM state diagram generated by Quartus Prime.

5.4 Phase Timer Module

The phase_timer module stores and updates the countdown value. It supports timer load, add, subtract, manual load, and one-second countdown operations. When timer_load is active, the timer is loaded with the value from the FSM. When timer_add is active, the timer increases by timer_value. When no special command is active, the timer decreases by one on each one-second tick if the current value is greater than zero.

The timer generates three important status signals. timer_expired is asserted when the timer reaches zero. timer_10sec_remaining indicates that the timer is less than or equal to ten seconds. timer_5sec_remaining indicates that the timer is less than five seconds. These signals are sent back to the FSM to support phase transitions and pedestrian request handling.

Table 5.2. Main phase timer signals.

Timer Signal	Function
timer_load	Load timer_count with timer_value
timer_add	Add timer_value to timer_count
timer_sub	Subtract timer_value from timer_count
manual_load	Load timer_count from switch input
tick_1s	Decrease timer_count by one every second
timer_expired	Indicates timer_count is zero
timer_10sec_remaining	Indicates timer_count is less than or equal to 10
timer_5sec_remaining	Indicates timer_count is less than 5

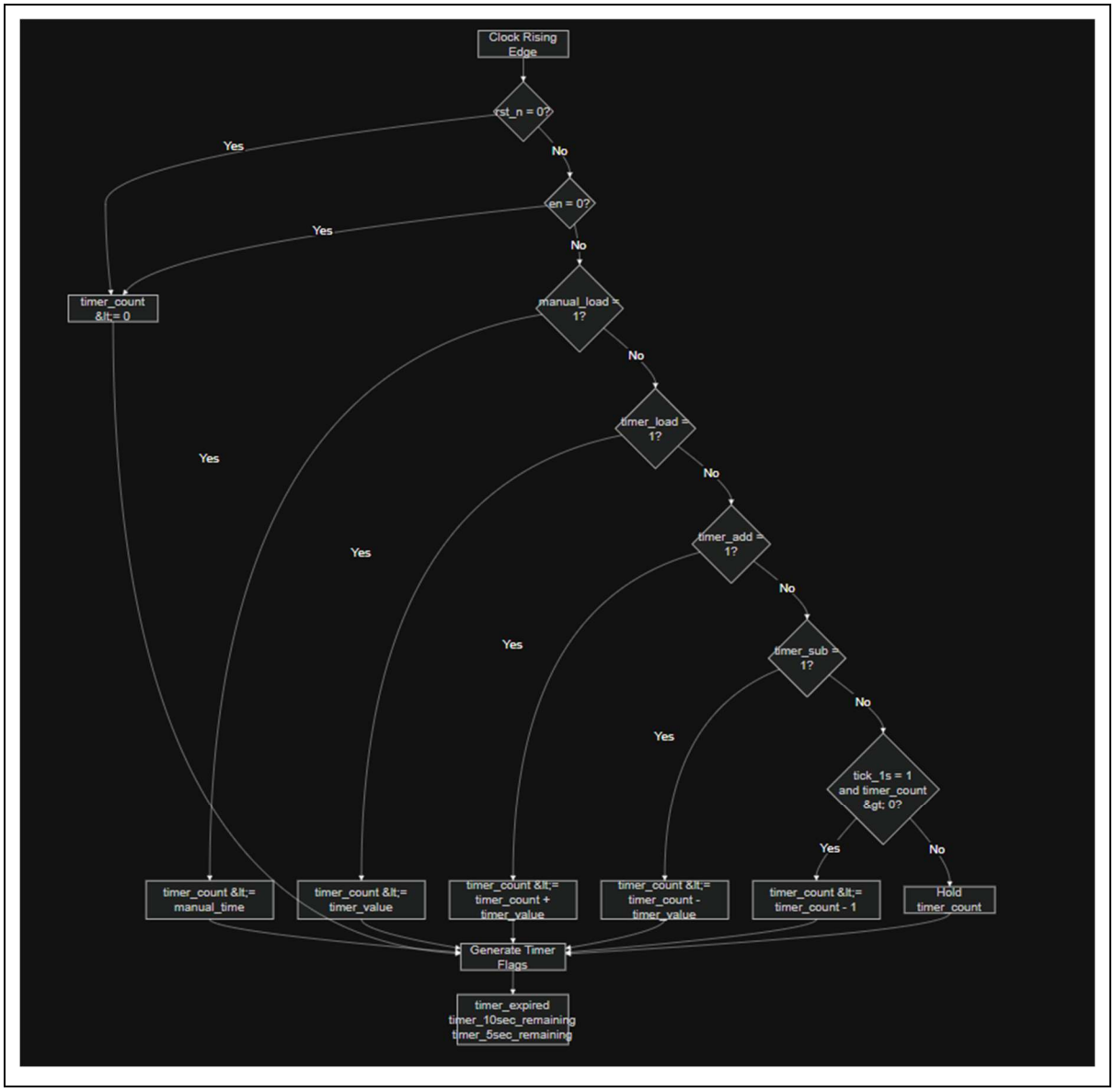


Figure 5.4. Phase timer operation flowchart.

5.5 Pedestrian Request Path

The pedestrian request path consists of the button_pulse module and the ped_req_reg module. The button_pulse module debounces the physical button input and produces a clean one-clock pulse. The ped_req_reg module stores this pulse as a pending request. The request remains active until the FSM accepts it and asserts req_clr.

This structure prevents the controller from missing a short button press. It also separates the physical button behavior from the traffic decision logic. The FSM only needs to read the stable ped_req signal instead of directly processing the raw button signal.

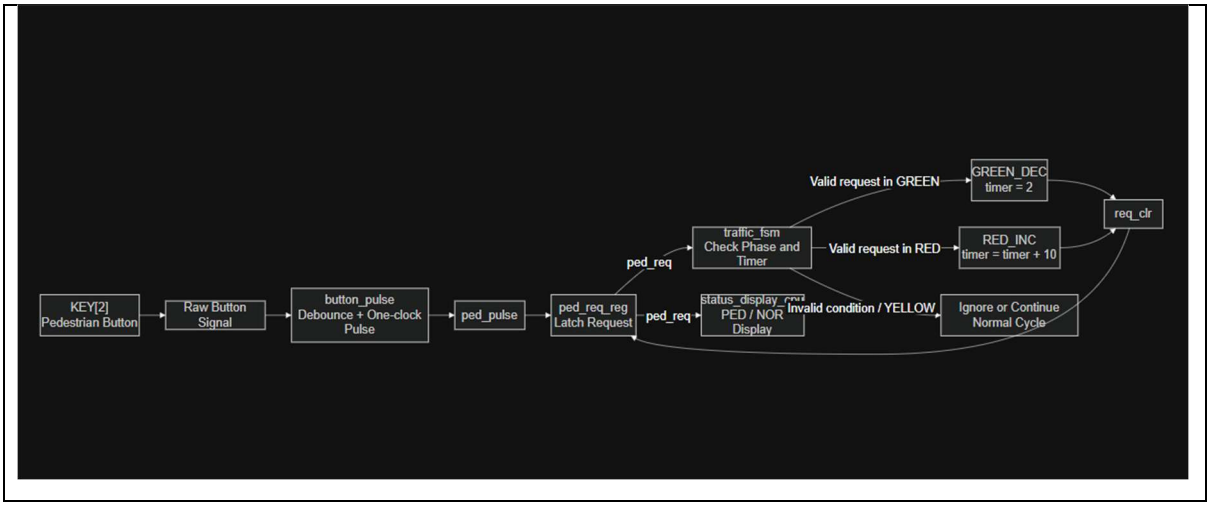


Figure 5.5. Pedestrian request signal path.

5.6 Display Modules

The display_7seg module converts the timer count into decimal digits and sends them to the hexled decoder. The hexled module converts a 4-bit decimal value into an active-low seven-segment pattern. This allows the countdown timer to be displayed on the FPGA board during simulation and hardware testing.

The status display module is used to show additional system information when the full set of seven-segment outputs is included. It can display the current phase as G, Y, or r, and the pedestrian request status as PED or NOR. This improves observability during hardware demonstration and debugging.

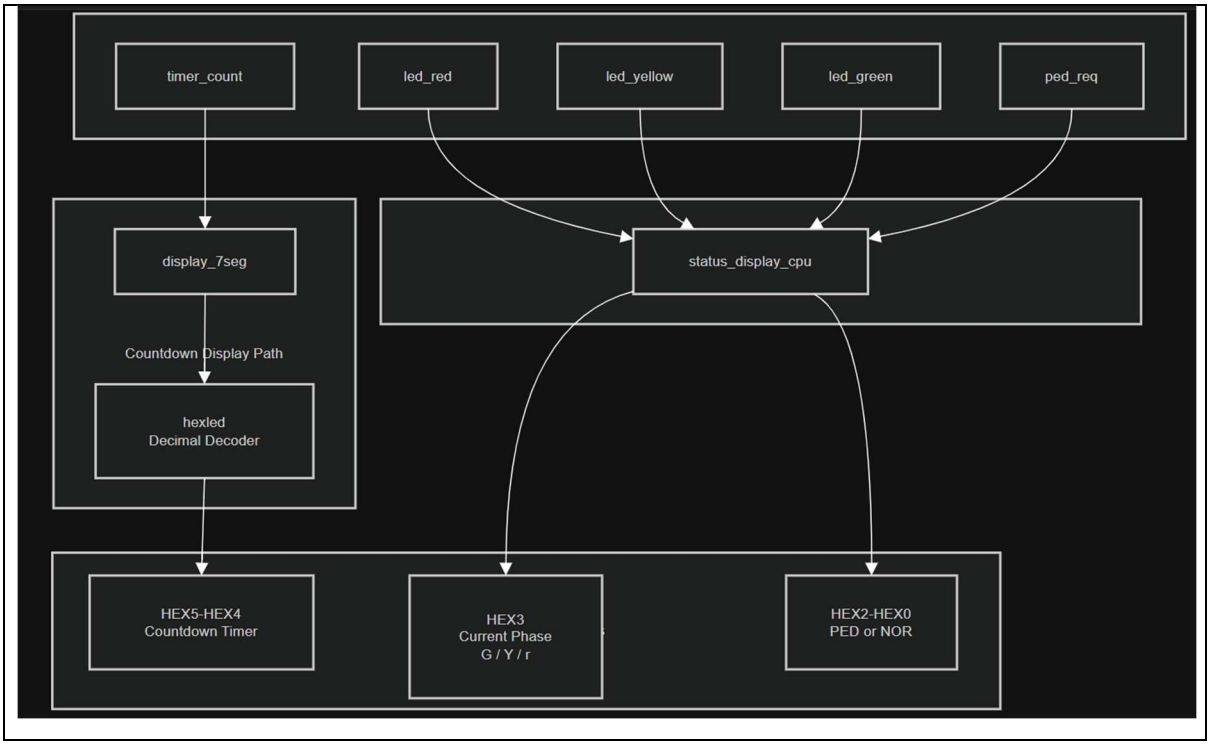


Figure 5.6. Seven-segment display usage.

5.7 Red Phase Buzzer Module

The red_phase_buzzer module provides audio feedback during the red phase. The module receives the red_active signal and the current timer_count value. When the red phase is active and the timer is greater than zero, it generates a beep pattern. The beep rate increases as the remaining red time becomes smaller.

The buzzer is divided into three levels. When more than 10 seconds remain, the buzzer beeps slowly. When 6 to 10 seconds remain, it beeps at a medium rate. When 1 to 5 seconds remain, it beeps quickly. This behavior is intended to imitate pedestrian crossing feedback found in practical traffic systems.

Table 5.3. Red phase buzzer behavior.

Timer Condition During Red	Buzzer Level	Behavior
timer_count > 10	1	Slow beep
6 <= timer_count <= 10	2	Medium beep
1 <= timer_count <= 5	3	Fast beep
Not red or timer_count = 0	0	Buzzer off

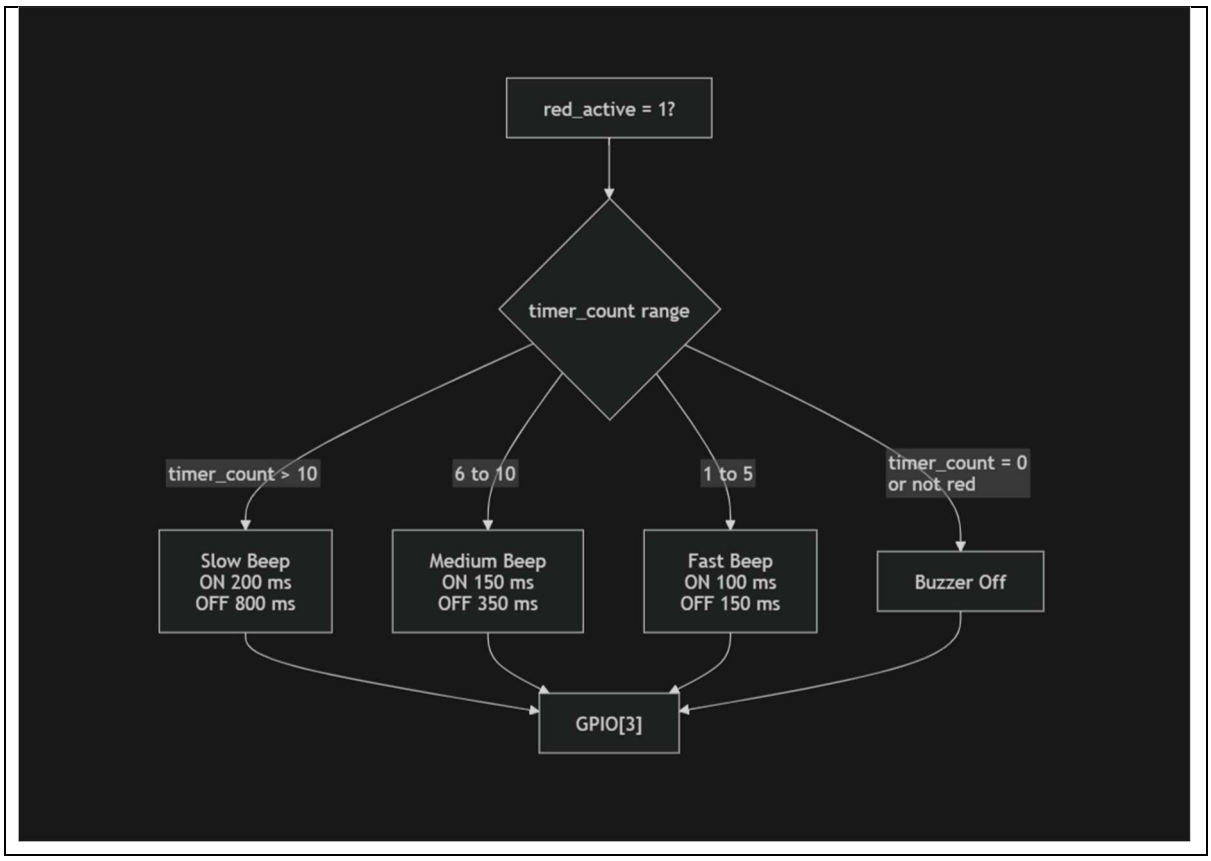


Figure 5.7. Buzzer timing behavior during the red phase.

Chapter 6. Simulation and Verification

6.1 Verification Methodology

Simulation was performed to verify the main behavior of the traffic light controller before and after hardware implementation. The verification focused on state transitions, timer operation, pedestrian request handling, display behavior, and buzzer output. This approach helped isolate design errors before downloading the bitstream to the FPGA board.

The verification process included module-level checks and system-level checks. The traffic FSM and phase timer were observed through ModelSim waveforms. The full controller was then tested using reset, enable, tick, and button input scenarios. Finally, hardware tests were performed on the DE10-Standard board to confirm real operation.

Table 6.1. Verification plan for the final design.

Verification Item	Purpose
Traffic FSM simulation	Verify phase transitions and request handling
Phase timer simulation	Verify load, add, manual load, and countdown behavior
Full controller simulation	Verify integration between FSM, timer, button, and display modules
Buzzer simulation	Verify buzzer level changes during red phase
Hardware demonstration	Confirm the design operates correctly on the FPGA board

6.2 Normal Traffic Cycle Verification

The first verification case checks the normal green-yellow-red sequence. After reset and enable, the controller should enter the green phase and load the timer with 15 seconds. After the timer expires, the controller should enter the yellow phase and load the timer with 3 seconds. After the yellow phase expires, the controller should enter the red phase and load the timer with 15 seconds. After the red phase expires, the cycle repeats.

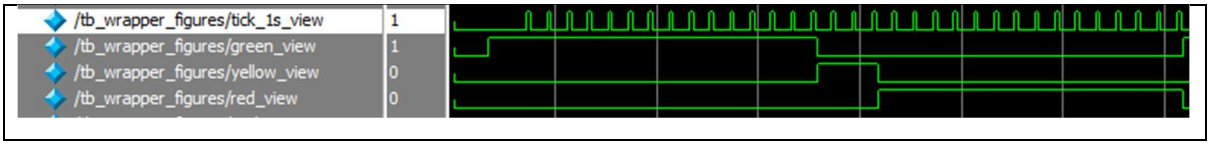


Figure 6.1. Waveform of the normal green-yellow-red cycle.

6.3 Pedestrian Request Verification

The pedestrian request behavior was verified in two cases. In the green phase, if the button is pressed while more than 10 seconds remain, the FSM should enter the GREEN_DEC state, clear the request, and force the timer to 2 seconds. In the red phase,

if the button is pressed when fewer than 5 seconds remain, the FSM should enter the RED_INC state, clear the request, and add 10 seconds to the timer.

These two test cases verify that the request latch, timer flags, and FSM transition conditions work together correctly. They also confirm that invalid or unsupported request conditions do not disturb the normal sequence.

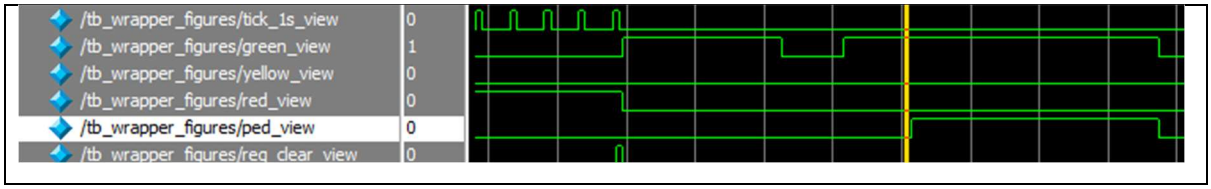


Figure 6.2. Waveform of pedestrian request during green phase.

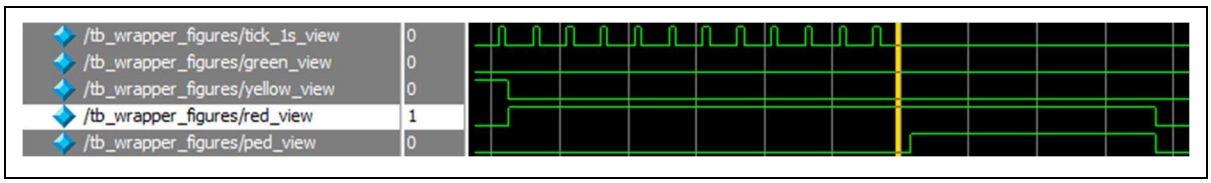


Figure 6.3. Waveform of pedestrian request during red phase.

6.4 Display and Buzzer Verification

The display and buzzer functions were also checked. The seven-segment outputs were observed to confirm that the countdown value changed according to the timer. The buzzer output was verified during the red phase. The expected behavior is that the buzzer is off outside the red phase and becomes active during red, with a faster beep pattern when the timer value becomes smaller.

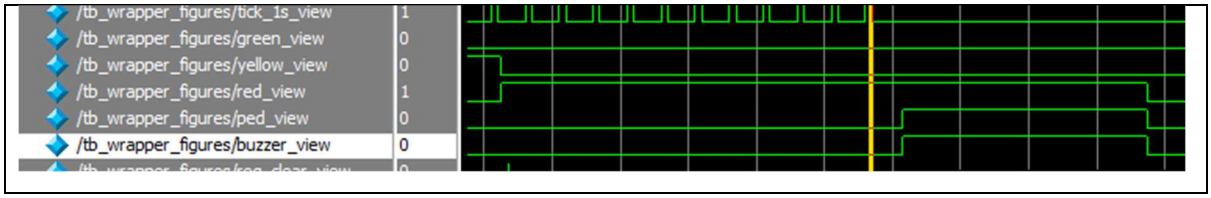


Figure 6.4. Waveform of buzzer output and timer count during red phase.

6.5 Verification Results Summary

The simulation and hardware verification results show that the system performs the required traffic light functions. The FSM correctly controls the phase sequence, the phase timer updates the countdown value, the pedestrian request is latched and cleared correctly, and the external outputs behave as expected during the hardware demonstration.

Table 6.2. Verification results summary.

Test Case	Expected Result	Status
Reset and enable	System starts from a known state and enters normal operation	Pass
Normal cycle	Green, yellow, and red phases occur in correct order	Pass
Green pedestrian request	Remaining green time is forced to 2 seconds	Pass
Red pedestrian request	Red timer is extended by 10 seconds	Pass
Countdown display	Timer value is displayed correctly	Pass
GPIO LED outputs	External LEDs follow the current phase	Pass
Buzzer output	Buzzer operates during red phase	Pass

```
# add wave *
# view structure
# .main_pane.structure.interior.cs.body.struct
# view signals
# .main_pane.objects.interior.cs.body.tree
# run -all
# ================================================
# Figure 6.1 - Normal GREEN-YELLOW-RED cycle
# ================================================
# [350000] PASS: Initial GREEN | R=0 Y=0 G=1 timer=15 ped=0
# [1850000] PASS: YELLOW after GREEN expired | R=0 Y=1 G=0 timer=3 ped=0
# [2150000] PASS: RED after YELLOW expired | R=1 Y=0 G=0 timer=15 ped=0
# [3650000] PASS: Back to GREEN after RED expired | R=0 Y=0 G=1 timer=15 ped=0
# ================================================
# Figure 6.2 - Pedestrian request during GREEN
# ================================================
# [4410000] PASS: Initial GREEN | R=0 Y=0 G=1 timer=15 ped=0
# [4550000] PASS: GREEN request accepted, timer forced to 2 | R=0 Y=0 G=1 timer=2 ped=1
# ================================================
# Figure 6.3 - Pedestrian request during RED
# ================================================
# [5510000] PASS: Initial GREEN | R=0 Y=0 G=1 timer=15 ped=0
# [7010000] PASS: YELLOW | R=0 Y=1 G=0 timer=3 ped=0
# [7310000] PASS: RED | R=1 Y=0 G=0 timer=15 ped=0
# [8410000] PASS: RED timer = 4 | R=1 Y=0 G=0 timer=4 ped=0
# [8550000] PASS: RED request accepted, timer extended to 14 | R=1 Y=0 G=0 timer=14 ped=1
# ================================================
# Figure 6.4 - Buzzer output during RED
# ================================================
# [9510000] PASS: Initial GREEN | R=0 Y=0 G=1 timer=15 ped=0
# [11010000] PASS: YELLOW | R=0 Y=1 G=0 timer=3 ped=0
# [11310000] PASS: RED | R=1 Y=0 G=0 timer=15 ped=0
# [11450000] Slow beep region
# [35950000] PASS: RED timer = 10 | R=1 Y=0 G=0 timer=10 ped=1
# [35950000] Medium beep region
# [54450000] PASS: RED timer = 5 | R=1 Y=0 G=0 timer=5 ped=1
# [54450000] Fast beep region
# ================================================
# ALL REPORT WAVEFORM TESTS COMPLETED
# ================================================
# ** Note: $stop    : C:/Vin_workspace/01_Projects/Quartus_Prime/TrafficlightIP_v1/svfiles/tb_wrapper_figures.sv(377)
#    Time: 68650 ns  Iteration: 0  Instance: /tb_wrapper_figures
# Break in Module tb_wrapper_figures at C:/Vin_workspace/01_Projects/Quartus_Prime/TrafficlightIP_v1/svfiles/tb_wrapper_figures.sv line 377
# A time value could not be extracted from the current line
# A time value could not be extracted from the current line
```

Figure 6.5. Simulation transcript of testbench.

Chapter 7. FPGA Synthesis and Quartus Results

7.1 Compilation Setup

The final design was synthesized using Intel Quartus Prime. The top-level entity for the working hardware version is wrapper. The design targets the DE10-Standard FPGA development board and uses the 50 MHz onboard clock. The source files are written in SystemVerilog and organized into controller, timer, display, button, and output modules.

Table 7.1. Quartus compilation setup.

Item	Description
Top-level entity	wrapper
Target board	DE10-Standard FPGA development board
HDL language	SystemVerilog
Main clock	CLOCK_50
Clock frequency	50 MHz
Main tool	Intel Quartus Prime
Simulation tool	ModelSim Intel FPGA Edition

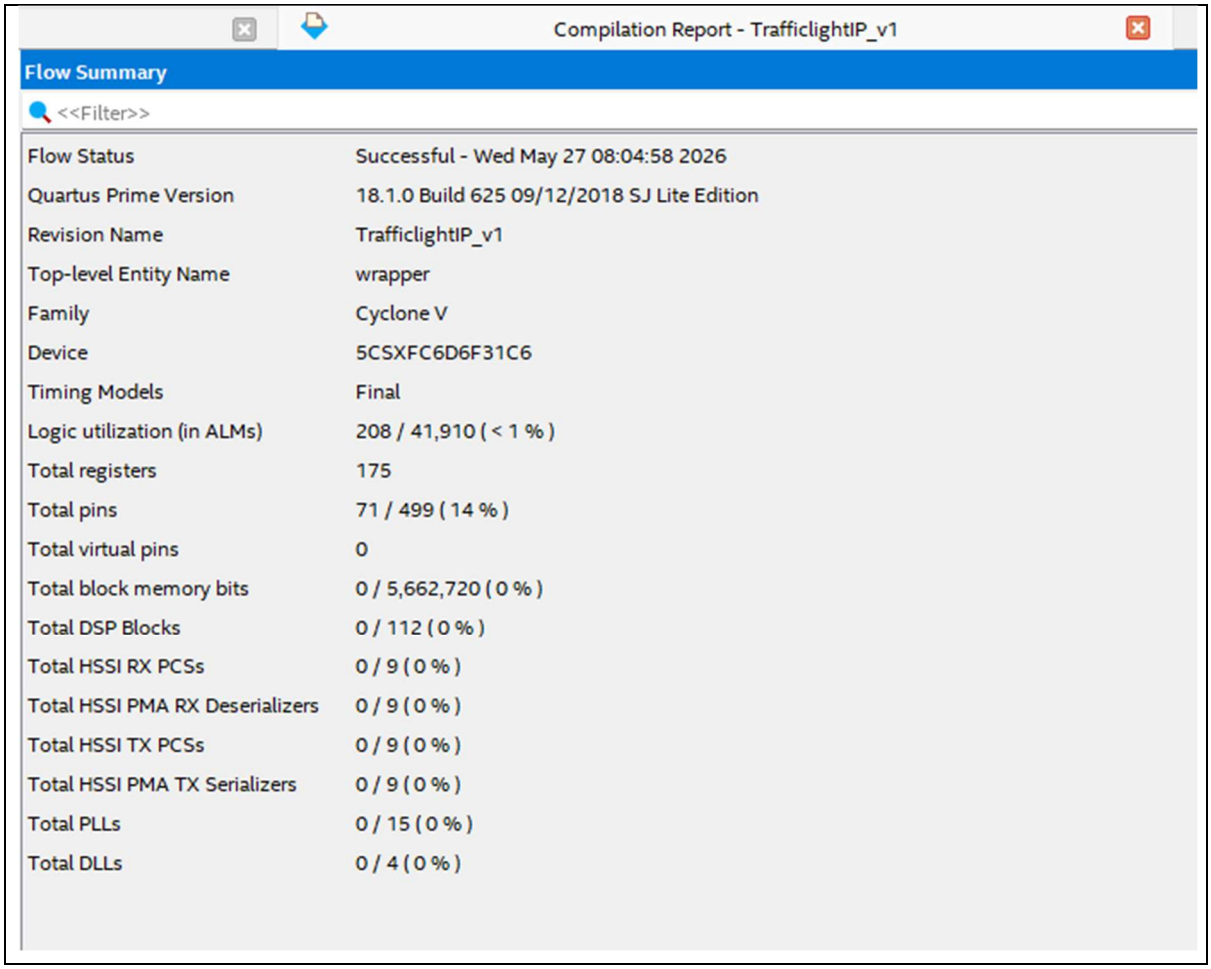


Figure 7.1. Quartus compilation summary.

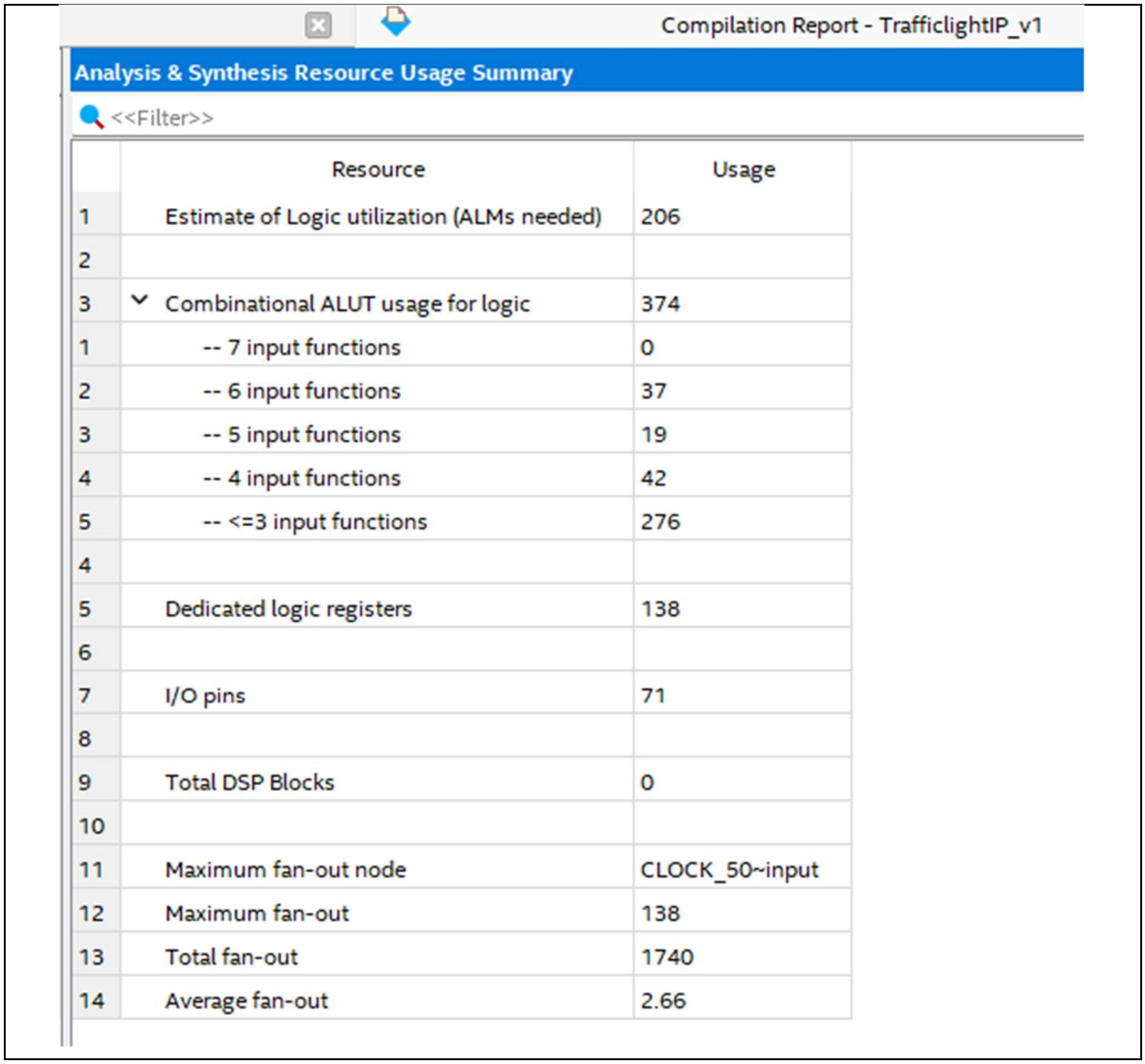
7.2 Resource Utilization

The Quartus resource report was reviewed to evaluate the hardware cost of the design. Since the final implementation is based on FSM control, timer registers, display logic, button filters, and a buzzer counter, the design mainly uses logic elements and flip-flops. It does not require large embedded memory blocks or DSP blocks.

The resource utilization reported by Quartus shows that the design uses only a small portion of the available FPGA resources.

Table 7.2. FPGA resource utilization summary.

Resource	Used	Available	Utilization
Logic elements / ALMs / LUTs	208	41,910	< 1 %
Registers	175	-	-
Total pins	71 top-level pins	499	14 %
Memory bits	0	5,662,720	0%
DSP blocks / multipliers	0	112	0%
PLLs	0	15	0%



	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	206
2		
3	Combinational ALUT usage for logic	374
1	-- 7 input functions	0
2	-- 6 input functions	37
3	-- 5 input functions	19
4	-- 4 input functions	42
5	-- <=3 input functions	276
4		
5	Dedicated logic registers	138
6		
7	I/O pins	71
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	CLOCK_50~input
12	Maximum fan-out	138
13	Total fan-out	1740
14	Average fan-out	2.66

Figure 7.2. Quartus resource utilization report.

7.3 RTL and State Machine Viewer

Quartus RTL Viewer and State Machine Viewer are useful for checking how the SystemVerilog modules are interpreted by the synthesis tool. The RTL Viewer can be

used to confirm the top-level hierarchy and connections between wrapper, Traffic_light_controller, traffic_fsm, phase_timer, and display modules. The State Machine Viewer can be used to check the traffic FSM state encoding and transition structure.

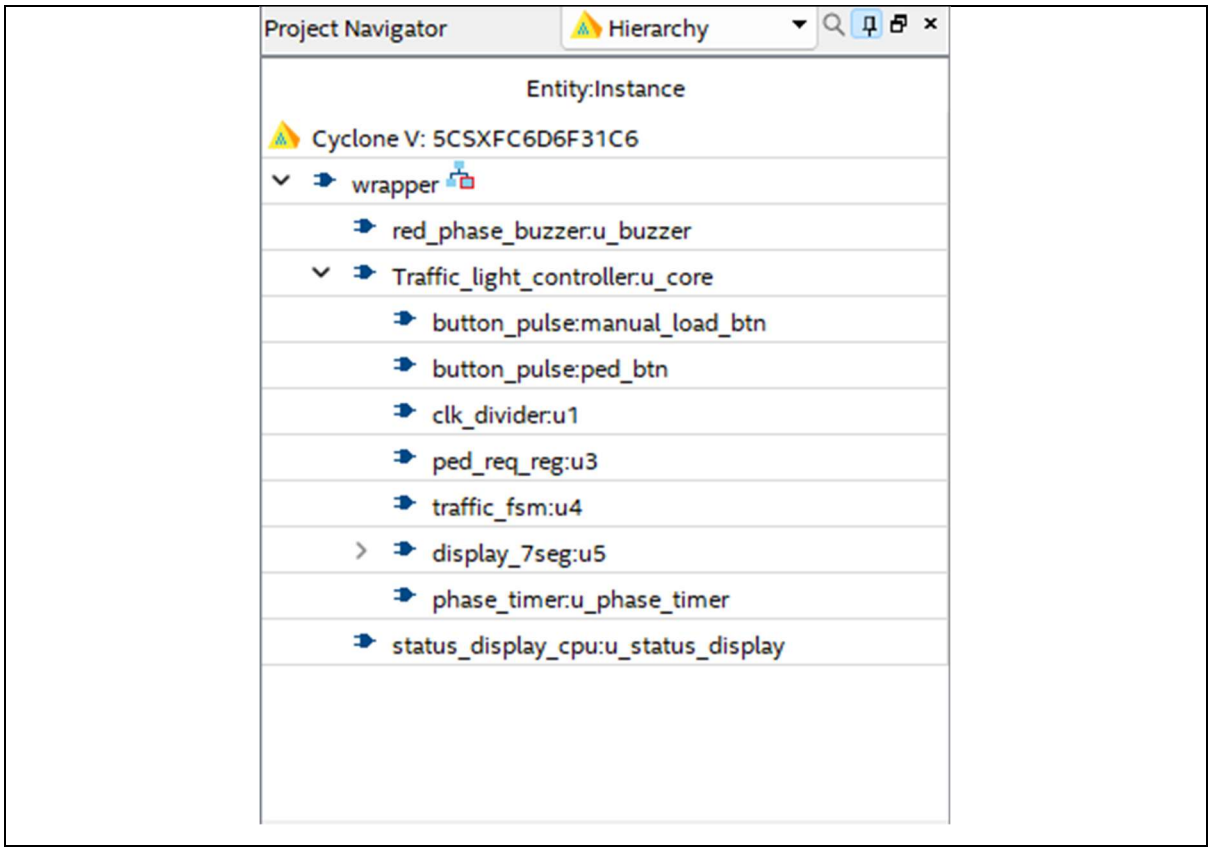


Figure 7.3. RTL Viewer of the final design hierarchy.

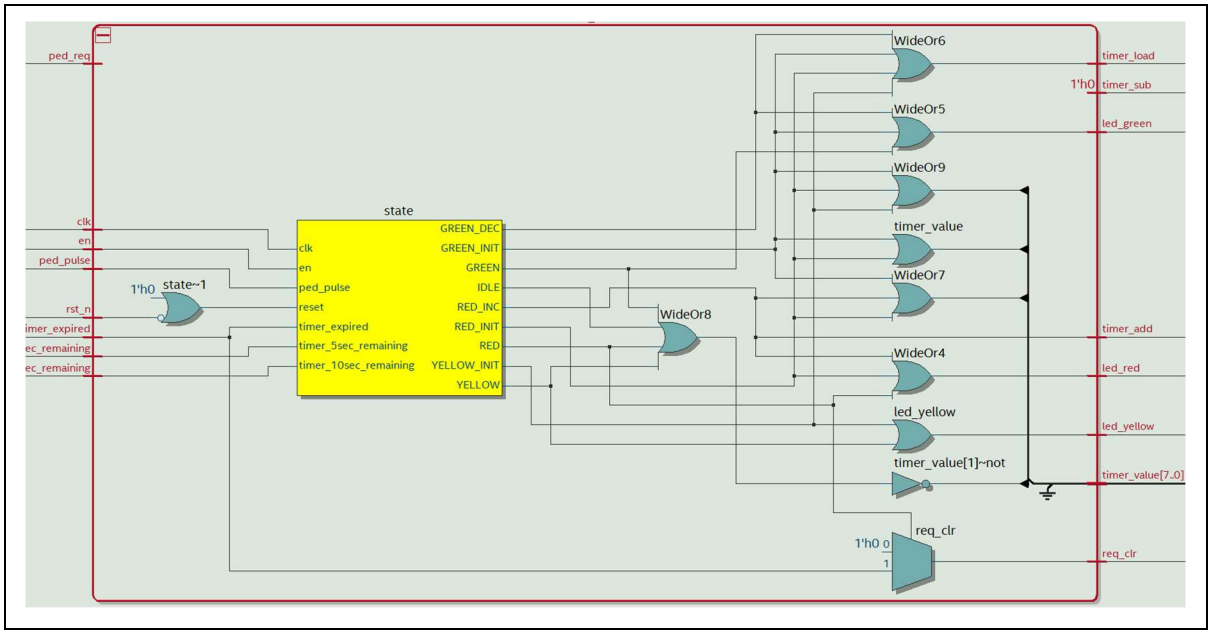


Figure 7.4. Quartus State Machine Viewer for traffic_fsm.

7.4 Timing Analysis

Timing analysis is required to confirm that the circuit can operate at the target 50 MHz clock frequency. The required clock period for a 50 MHz clock is 20 ns. A Synopsys Design Constraints file should define CLOCK_50 as the main clock. After compilation, the setup and hold timing reports should be checked.

If timing is met, the design can be considered suitable for the target board frequency. If timing is not met, the critical paths should be reviewed and optimized. For this project, the logic is relatively simple, so timing is expected to be manageable after applying the correct clock constraint.

Table 7.3. Timing analysis summary.

Timing Item	Result
Target clock	CLOCK_50
Target frequency	50 MHz
Required period	20 ns
Worst setup slack	See Figure 7.5
Worst hold slack	See Figure 7.5
Timing status	Verified by compilation and hardware test

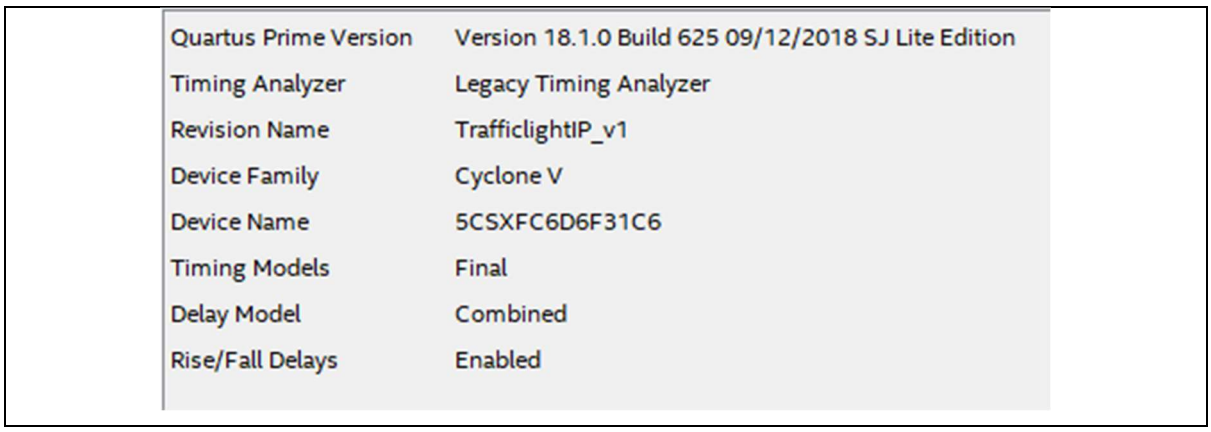


Figure 7.5. Timing Analyzer summary.

7.5 Compilation Warnings

Quartus may report warnings during compilation. These warnings should be reviewed to determine whether they affect functionality. Some warnings are harmless, such as processor count settings or constant seven-segment output segments. Other warnings, such as missing pin assignments or missing timing constraints, must be resolved before final hardware testing.

Table 7.4. Compilation warning analysis.

Warning	Cause	Action
NUM_PARALLEL_PROCESSORS not specified	Quartus project setting	Optional; does not affect hardware function
Truncated value in display_7seg	Division/modulo result assigned to 4-bit digit signals	Use temporary signals or explicit casting if needed
Stuck HEX pins	Some seven-segment segments are constant for fixed characters	Acceptable if display behavior is correct
Unused input pins	Some KEY or SW inputs are reserved or unused	Acceptable or reserved for extension
No exact pin location assignment	Physical pins not assigned	Must be fixed before board test
SDC file not found	Missing clock constraint file	Add SDC file for accurate timing analysis

Chapter 8. Hardware Implementation and Demonstration

8.1 Hardware Setup

The hardware demonstration was performed using the DE10-Standard FPGA development board and an external breadboard circuit. The breadboard circuit contains three traffic LEDs and a buzzer driver. The FPGA board provides the control signals through GPIO pins, while the external circuit displays the traffic light state and produces the buzzer feedback.

The three LEDs represent the red, yellow, and green traffic lights. Each LED is connected through a current-limiting resistor. The buzzer is connected through a driver circuit so that the FPGA GPIO pin only provides a low-current control signal. The FPGA board and breadboard circuit share a common ground.

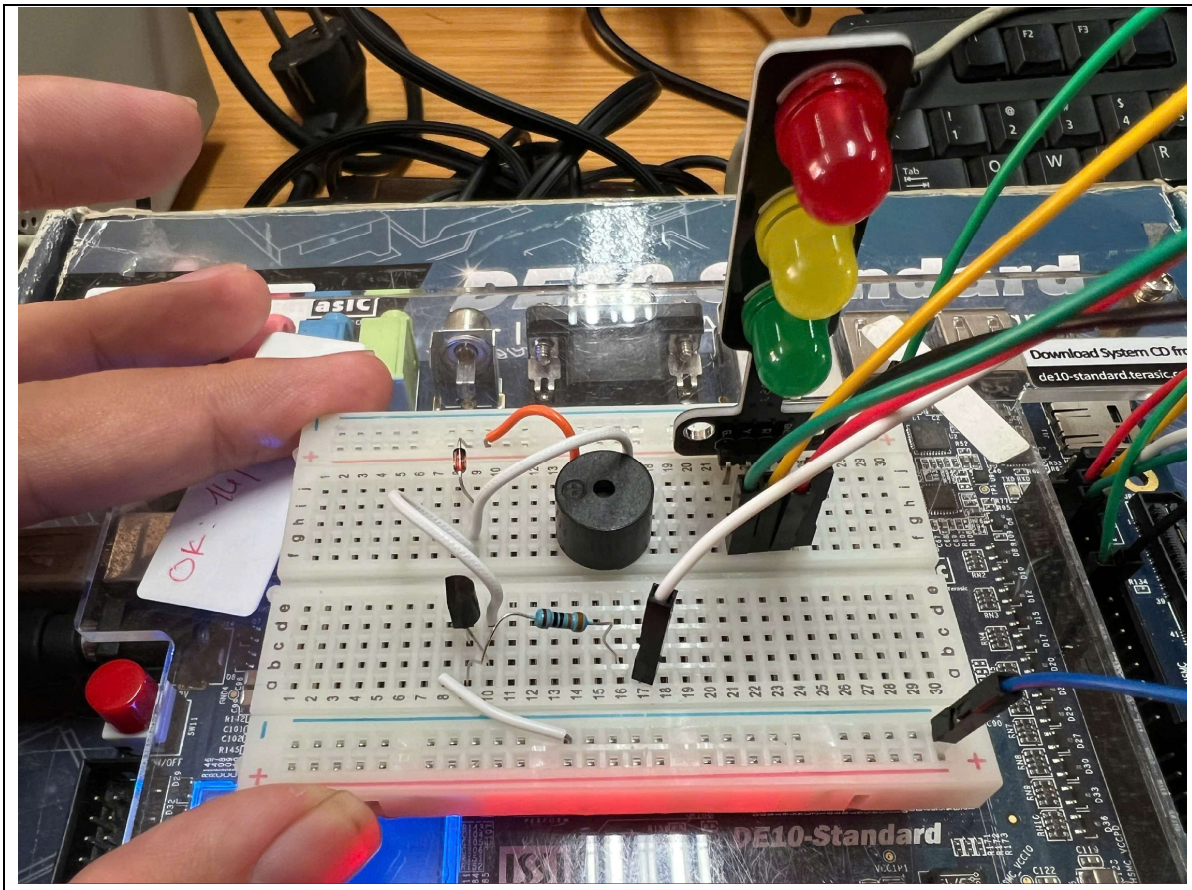


Figure 8.1. Breadboard circuit for external LEDs and buzzer driver.

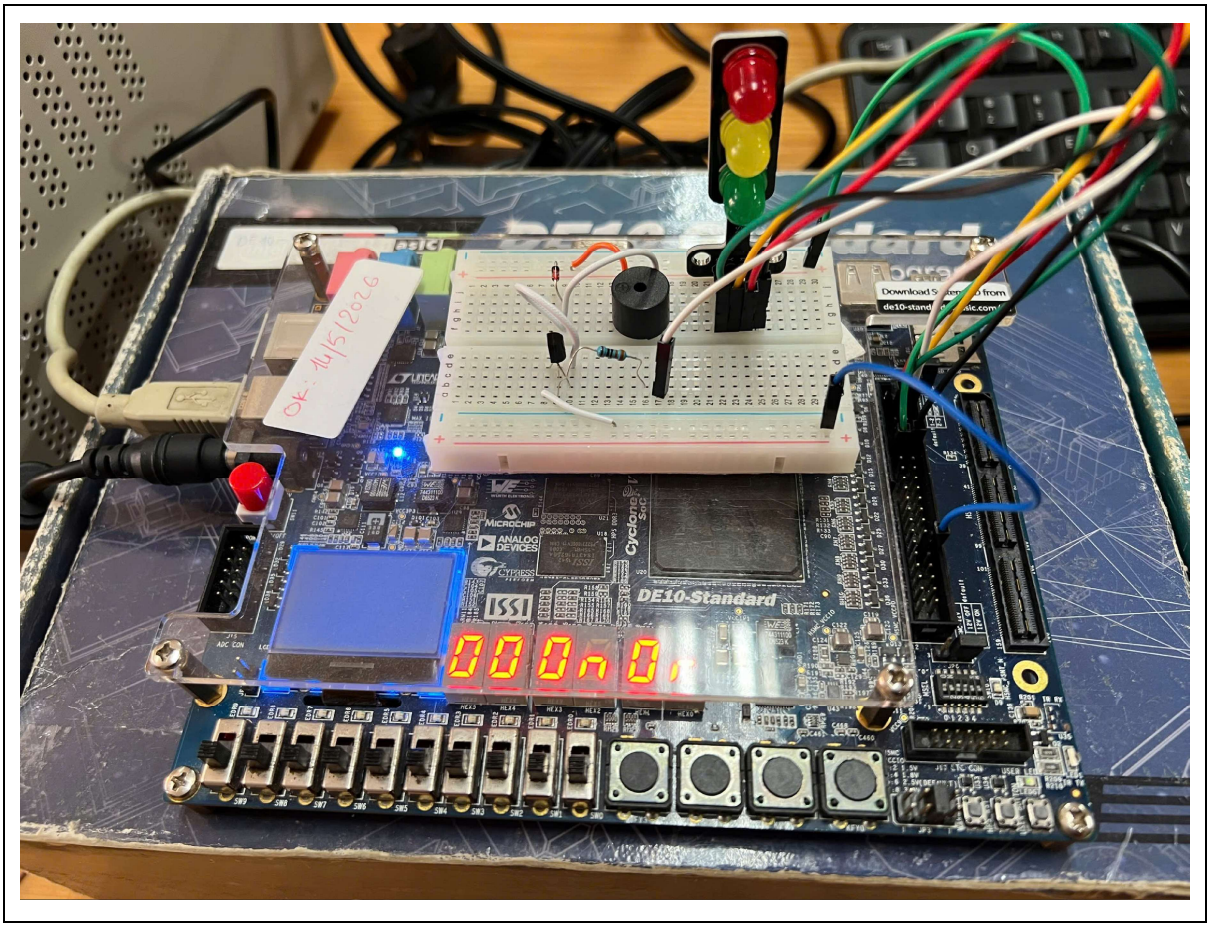


Figure 8.2. Complete hardware setup with DE10-Standard board.

8.2 GPIO and External Circuit Connection

The GPIO outputs are used to connect the FPGA logic to the external hardware. GPIO[0] controls the red LED, GPIO[1] controls the yellow LED, GPIO[2] controls the green LED, and GPIO[3] controls the buzzer driver. This mapping allows the traffic light outputs to be observed through external components instead of only through onboard LEDs.

Table 8.1. External GPIO connection for hardware demonstration.

GPIO Pin	External Circuit Function
GPIO[0]	Red traffic LED
GPIO[1]	Yellow traffic LED
GPIO[2]	Green traffic LED
GPIO[3]	Buzzer driver input

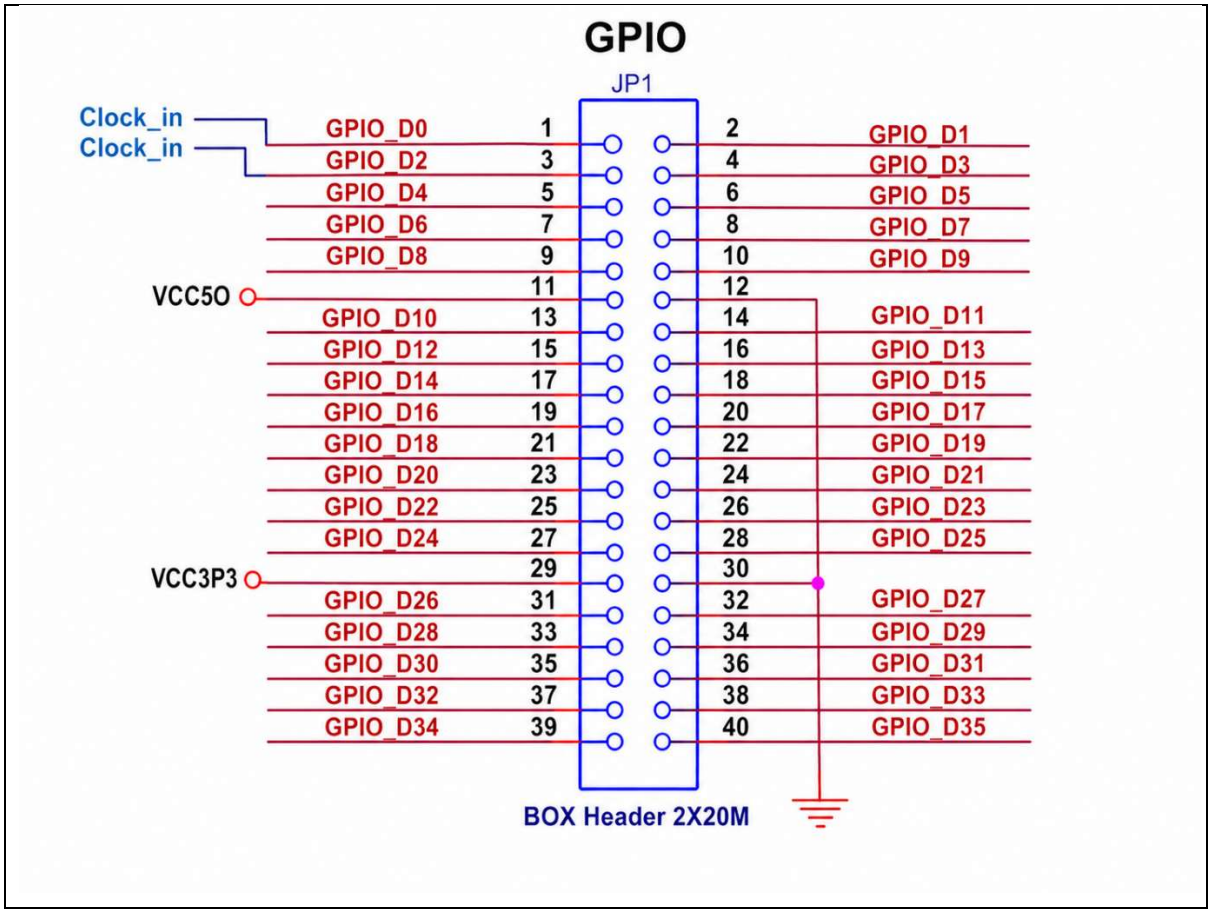


Figure 8.3. GPIO pinout of FPGA DE-10 Standard.

8.3 Hardware Test Results

The hardware test was performed by programming the FPGA board with the final bitstream and observing the system operation. The main test cases include reset and enable, normal traffic cycle, pedestrian request during green, pedestrian request during red, countdown display, external LED outputs, and buzzer output during the red phase.

Table 8.2. Hardware test results.

Test Case	Expected Result	Observed Result	Status
Reset and enable	System starts correctly after reset and enable	Entered normal operation after reset and SW[9] enable.	Pass
Normal green phase	Green LED active and timer counts down	GPIO[2] was active and countdown decreased normally.	Pass
Green to yellow transition	Yellow LED active after green expires	Changed from green to yellow after the green countdown.	Pass
Yellow to red transition	Red LED active after yellow expires	Changed from yellow to red after	Pass

		the yellow countdown.	
Pedestrian request during green	Green timer is forced to 2 seconds	Valid green request shortened the remaining green time.	Pass
Pedestrian request during red	Red timer is extended by 10 seconds	Valid red request extended the red phase.	Pass
Buzzer feedback	Buzzer beeps during red phase	GPIO[3] drove the buzzer during red-phase feedback.	Pass

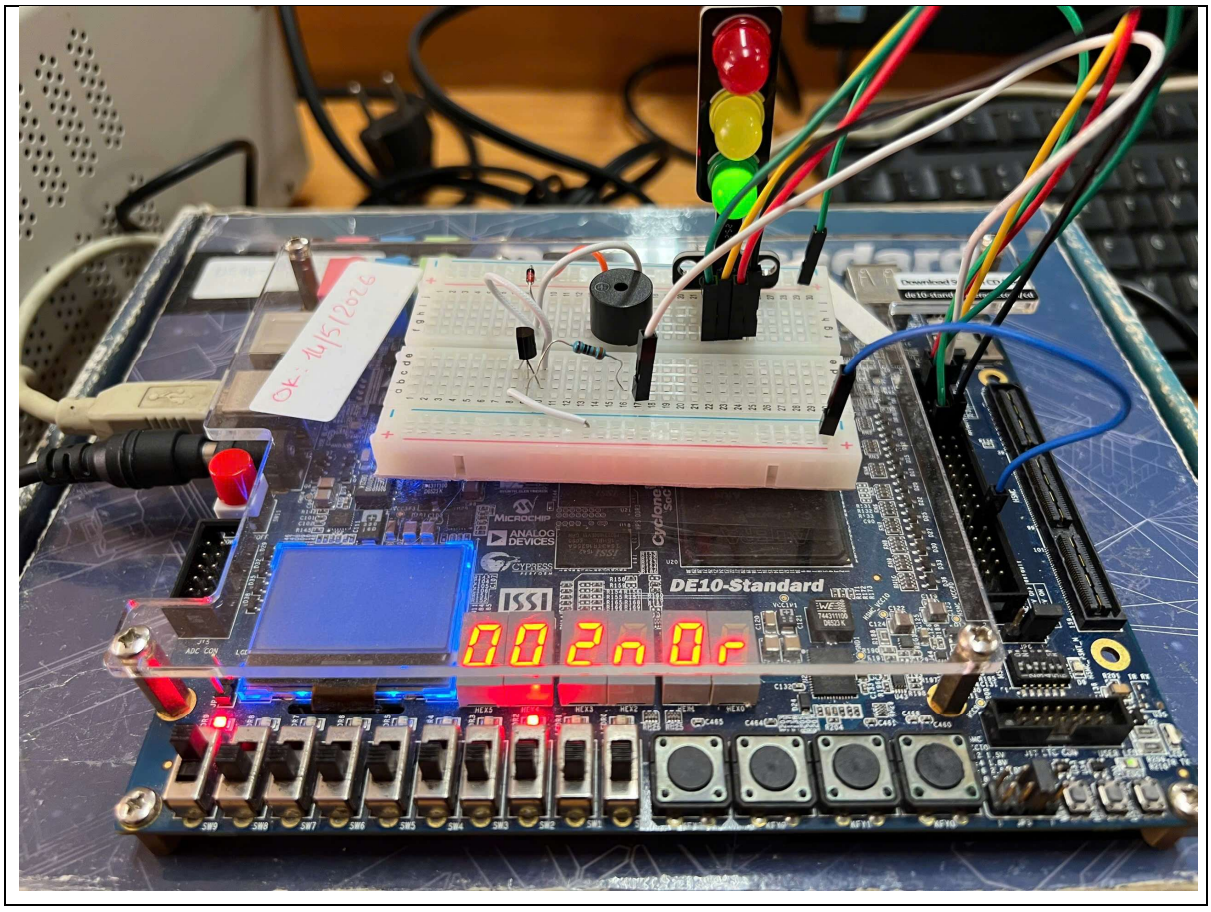


Figure 8.4. Green phase hardware demonstration.

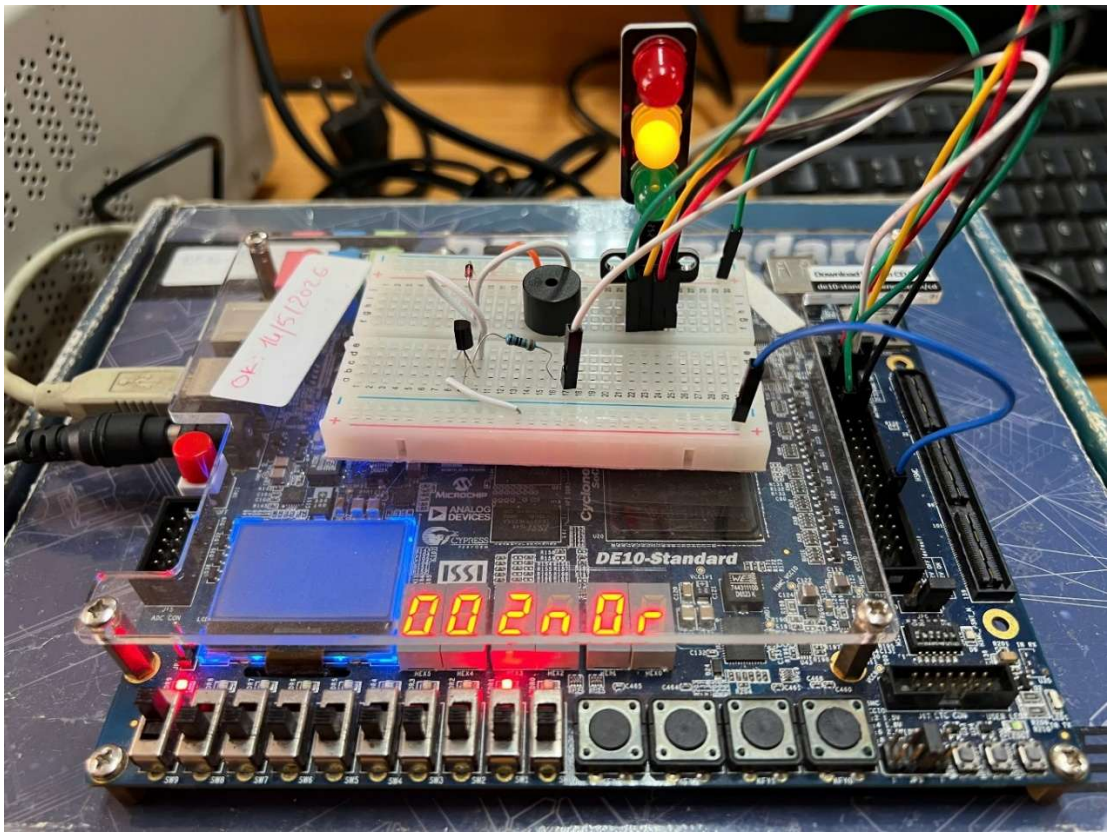


Figure 8.5. Yellow phase hardware demonstration.

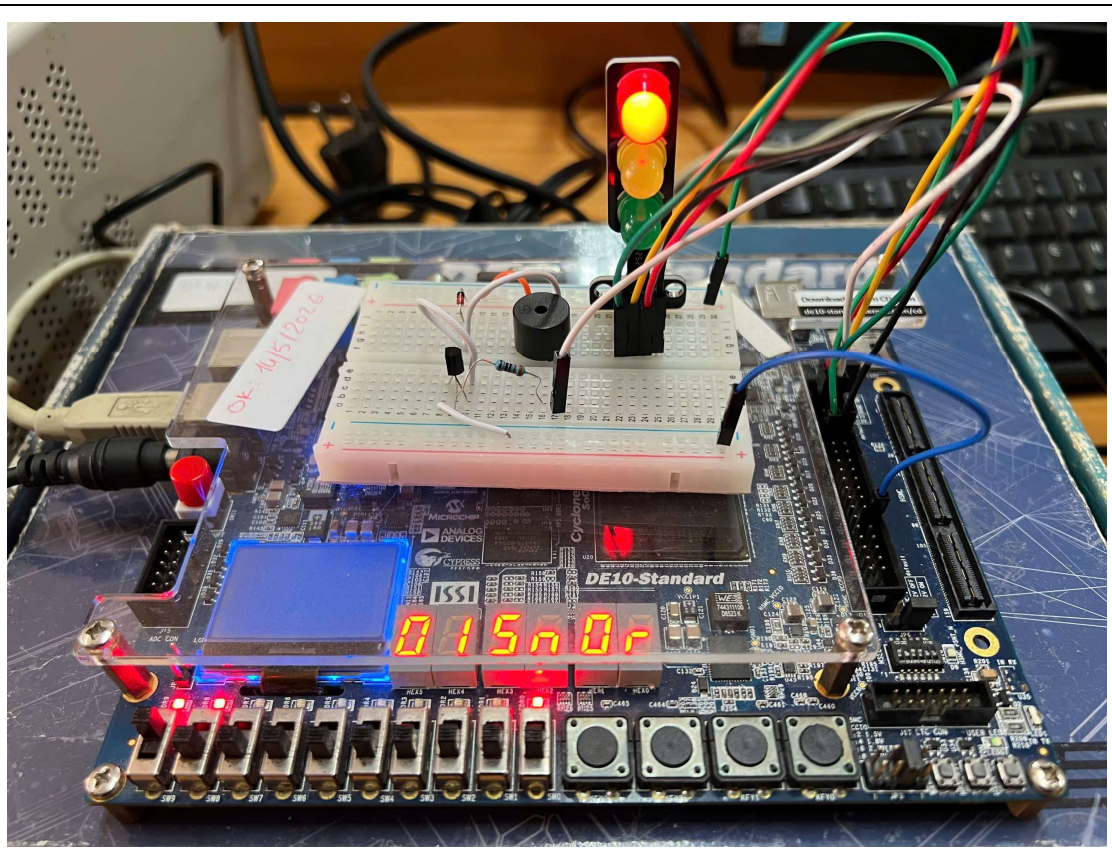


Figure 8.6. Red phase hardware demonstration with buzzer.

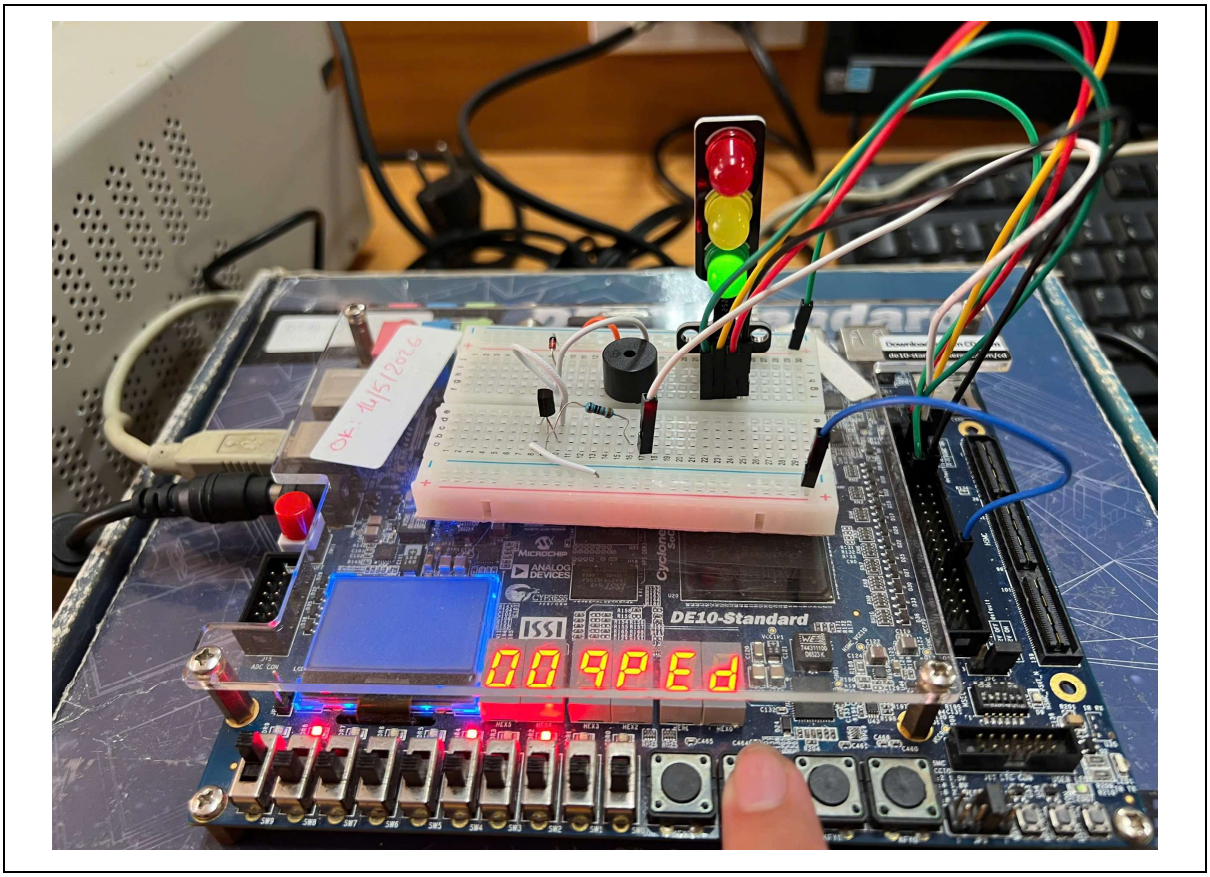


Figure 8.7. Pedestrian request test on hardware.

8.4 Evaluation Summary

The hardware demonstration confirms whether the synthesized design behaves correctly on the actual FPGA board. The most important evaluation point is that the real traffic light sequence matches the simulation results. The external LEDs should follow the FSM outputs, the countdown should update at approximately one-second intervals, and the buzzer should operate during the red phase.

The successful hardware test shows that the project is not only a simulation exercise but also a working FPGA implementation. It verifies the complete chain from SystemVerilog design, simulation, synthesis, pin assignment, and breadboard-level hardware integration.

Demonstration video links:

Normal test: https://youtu.be/FeruBrl-f3E

Pedestrian request test: https://youtu.be/vQW0ryRxSX8

Timer manual set test: https://youtu.be/0m8TjZSndpA

Chapter 9. Conclusion

9.1 Conclusion

This capstone project successfully implements an enhanced FPGA-based traffic light controller using SystemVerilog. The final working system is based on a finite state machine architecture and is demonstrated on the DE10-Standard FPGA board. The controller supports the normal green-yellow-red traffic sequence, pedestrian request handling, countdown timing, external LED output, and red-phase buzzer feedback.

The project combines several important digital design concepts, including FSM control, button debouncing, request latching, countdown timer design, seven-segment display driving, GPIO interfacing, and external hardware integration. The design was verified through simulation and then tested on the FPGA board with a breadboard LED and buzzer circuit.

9.2 Project Achievements

The main achievement of the project is the successful implementation of a working traffic light controller on FPGA. The system performs the normal traffic sequence and responds to pedestrian requests according to the specified timing conditions. The phase timer provides real-time countdown behavior, while the GPIO outputs drive external traffic LEDs and the buzzer driver.

Another important achievement is the hardware demonstration. The final design was not limited to simulation; it was downloaded to the DE10-Standard board and tested with external components. This confirms that the SystemVerilog design, Quartus synthesis, pin assignments, and breadboard hardware connection all work together as a complete digital system.

Table 9.1. Summary of project achievements.

Achievement	Description
FSM traffic control	Implemented green, yellow, and red phase sequence
Pedestrian request handling	Shortens green or extends red based on request timing
Countdown timer	Provides phase timing and status flags
External outputs	Drives traffic LEDs and buzzer through GPIO
Hardware demonstration	Verified on DE10-Standard board with breadboard circuit

9.3 Limitations and Future Work

The current project controls one traffic direction only. It does not implement a complete two-way intersection with coordinated traffic phases. The pedestrian request is generated by a push button rather than by an automatic sensor. The buzzer output is also a simple on/off beep pattern, which is suitable for an active buzzer or driver circuit but does not generate a full audio-frequency PWM tone.

Future improvements could include extending the system to a two-direction intersection, adding a more advanced pedestrian crossing mode, improving the buzzer output using PWM audio generation, adding automatic pedestrian or vehicle detection, and integrating manual timing control more completely. A CPU-based version of the controller may also be revisited in the future after additional debugging time.

Table 9.2. Limitations and future improvements.

Limitation	Possible Future Improvement
Single-direction traffic controller	Extend to a two-direction road intersection
Button-based request only	Add sensor-based pedestrian detection
Simple buzzer pattern	Use PWM to generate clearer audio tone
Limited manual time feature	Complete manual load and configuration control
FSM-only final implementation	Revisit CPU-assisted architecture after further debugging

References

[1] Terasic Technologies, DE10-Standard User Manual, Terasic Inc., 2017.

[2] Intel Corporation, Intel Quartus Prime Standard Edition User Guides, Intel FPGA Documentation, 2024.

[3] Siemens EDA, ModelSim User's Manual, Siemens Digital Industries Software, 2024.

[4] IEEE Computer Society, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language, IEEE Std. 1800-2017, 2018.

[5] S. Brown and Z. Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd ed. New York, NY, USA: McGraw-Hill, 2014.

[6] M. M. Mano and M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Boston, MA, USA: Pearson, 2013.

[7] D. M. Harris and S. L. Harris, Digital Design and Computer Architecture, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013.

[8] P. P. Chu, FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version. Hoboken, NJ, USA: Wiley, 2008.

[9] S. Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2003.

[10] Intel Corporation, Intel Quartus Prime Standard Edition User Guide: Timing Analyzer, Intel FPGA Documentation, 2024.

Appendices

Appendix A. Source File List

Table A.1. SystemVerilog source file list.

No.	Source File	Function
1	Traffic_light_controller.sv	Main controller core
2	wrapper.sv	Board-level top module
3	traffic_fsm.sv	Traffic phase FSM
4	phase_timer.sv	Countdown timer
5	ped_req_reg.sv	Pedestrian request latch
6	button_pulse.sv	Button debounce and pulse generator
7	clk_divider.sv	One-second tick generator
8	display_7seg.sv	Countdown display module
9	hexled.sv	Seven-segment digit decoder
10	status_display_cpu.sv	Status display module
11	red_phase_buzzer.sv	Red-phase buzzer module

Appendix B. Additional Simulation and Quartus Results

Figure E.1. ModelSim waveform of the final wrapper-level simulation for the enhanced traffic light controller.

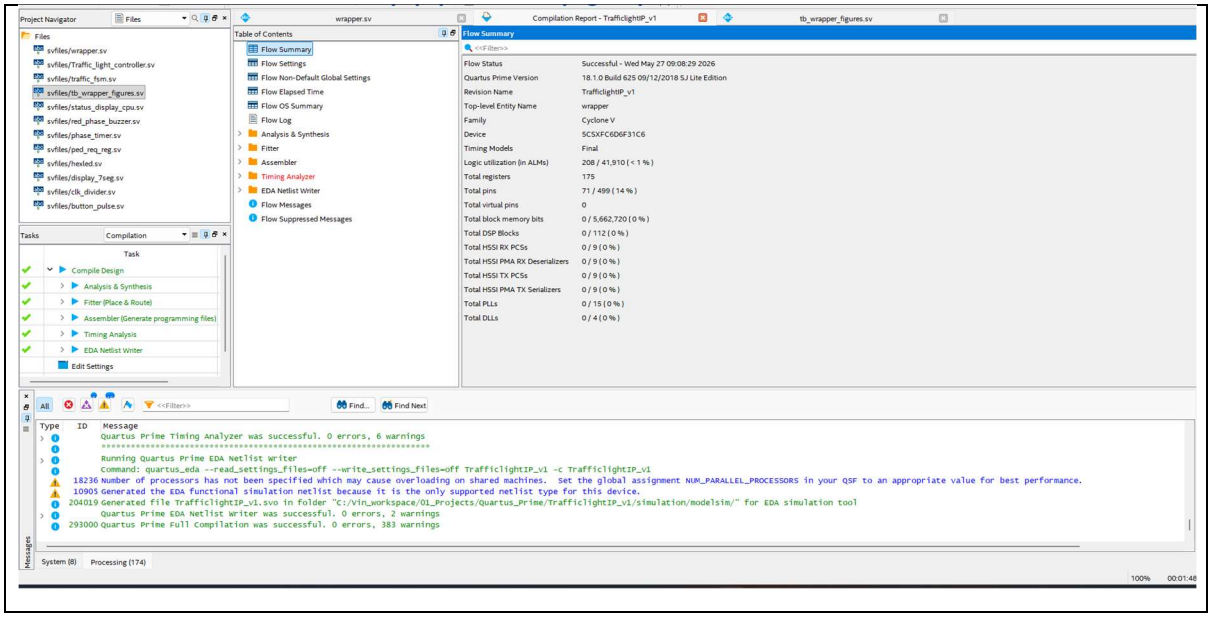


Figure F.1. Successful Quartus Prime compilation result and resource summary for the final wrapper top-level design.

-The End-